Universidad Autónoma de Bucaramanga · UNAB

Informe de evidencias · Partes 1 y 2

API REST tipo Facebook con Laravel

Publicaciones, imágenes, comentarios y likes · Autenticación, paginación, búsqueda, correo, versionado y documentación

Autor	Juan Camilo Calderon (trabajo individual)
Materia	Arquitectura y Desarrollo Backend
Institución	Universidad Autónoma de Bucaramanga - UNAB
Docente	Fabian Enrique Suarez Carvajal
Repositorio	https://github.com/CamiloCalder0n/-facebook-api (Parte 1 en el tag parte-1)
Fecha	12 de octubre de 2026

Contenido

Parte	Taller	Páginas
Parte 1	Creación de una API REST tipo Facebook con Laravel (taller del 8 de octubre de 2026)	2 – 25
Parte 2	Retos Avanzados de API REST con Laravel (Sanctum, paginación, likes por usuario, búsqueda, correo, versionado y Scribe)	26 – 41

Cada parte conserva su propia portada, sus figuras numeradas y su checklist final. La numeración de páginas es continua en todo el documento.

Universidad Autónoma de Bucaramanga · UNAB

Evidencias del taller

Creación de una API REST tipo Facebook con Laravel

Autor	Juan Camilo Calderon (trabajo individual)
Materia	Arquitectura y Desarrollo Backend
Institución	Universidad Autónoma de Bucaramanga - UNAB
Docente	Fabian Enrique Suarez Carvajal
Fecha	8 de octubre de 2026
Repositorio	https://github.com/CamiloCalder0n/-facebook-api

2. Entorno y configuración

Salidas de terminal registradas el 8 de octubre de 2026 a las 14:21 (archivo de evidencia de terminal).

Servidor respondiendo

```
$ curl -I http://127.0.0.1:8000  (servidor php artisan serve)
HTTP 200
```

Enlace simbólico de storage

```
$ ls -la public/storage
lrwxr-xr-x@ 1 camilocalderon  staff  53 Oct  8 11:45 public/storage
    -> /Users/camilocalderon/facebook-api/storage/app/public
```

Migraciones (tablas posts, post_images y comments)

```
$ php artisan migrate:status

 Migration name .. Batch / Status
 0001_01_01_000000_create_users_table .. [1] Ran
 0001_01_01_000001_create_cache_table .. [1] Ran
 0001_01_01_000002_create_jobs_table .. [1] Ran
 2026_10_08_164530_create_personal_access_tokens_table .. [1] Ran
 2026_10_08_165109_create_posts_table .. [1] Ran
 2026_10_08_165110_create_post_images_table .. [1] Ran
 2026_10_08_165111_create_comments_table .. [1] Ran
```

Rutas de la API

```
$ php artisan route:list --path=api

  GET|HEAD  api/posts ................ PostController@index
  POST      api/posts ................ PostController@store
  GET|HEAD  api/posts/{id} ........... PostController@show
  DELETE    api/posts/{id} ........... PostController@destroy
  GET|HEAD  api/posts/{id}/comments .. CommentController@index
  POST      api/posts/{id}/comments .. CommentController@store
  POST      api/posts/{id}/like ...... PostController@like

  Showing [7] routes
```

Pruebas automáticas (15/15 en verde)

```
$ php artisan test
{"tool":"phpunit","result":"passed","tests":15,"passed":15,"assertions":88,"duration_ms":139}
```

Servidor en ejecución

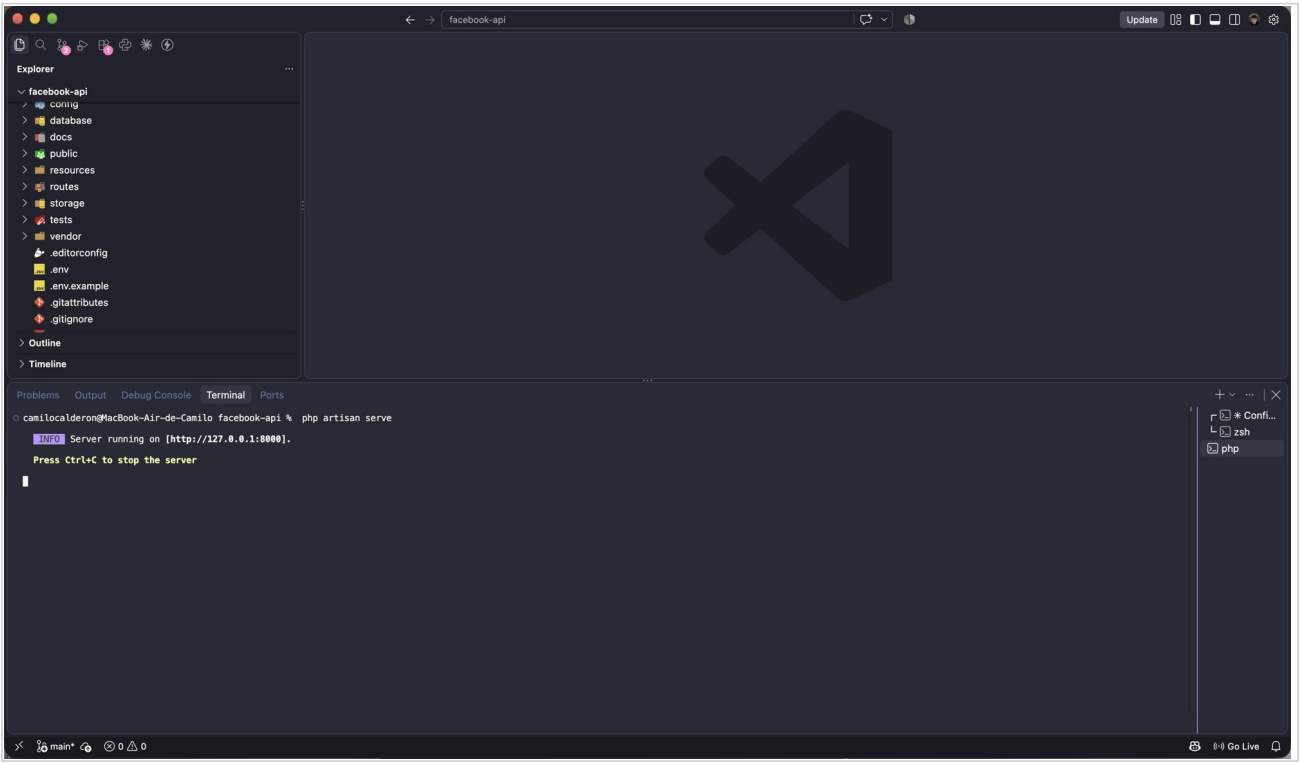


Figura 1. Servidor iniciado con "php artisan serve" desde la terminal de VS Code: "Server running on [http://127.0.0.1:8000]".

3. Pruebas de éxito en Postman

Cada prueba se ejecutó desde la colección **Facebook API - Taller Backend**, con la variable **{{base_url}}** = http://127.0.0.1:8000 y el header **Accept: application/json**. Se muestran en el orden de ejecución del flujo (crear, leer, comentar, dar like, eliminar y verificar).

3.1 Crear publicación con imágenes e imagen en el navegador

Petición: POST {{base_url}}/api/posts (form-data: title, content, images[] ×2)

Código esperado: 201 Created | **Código obtenido:** 201 Created

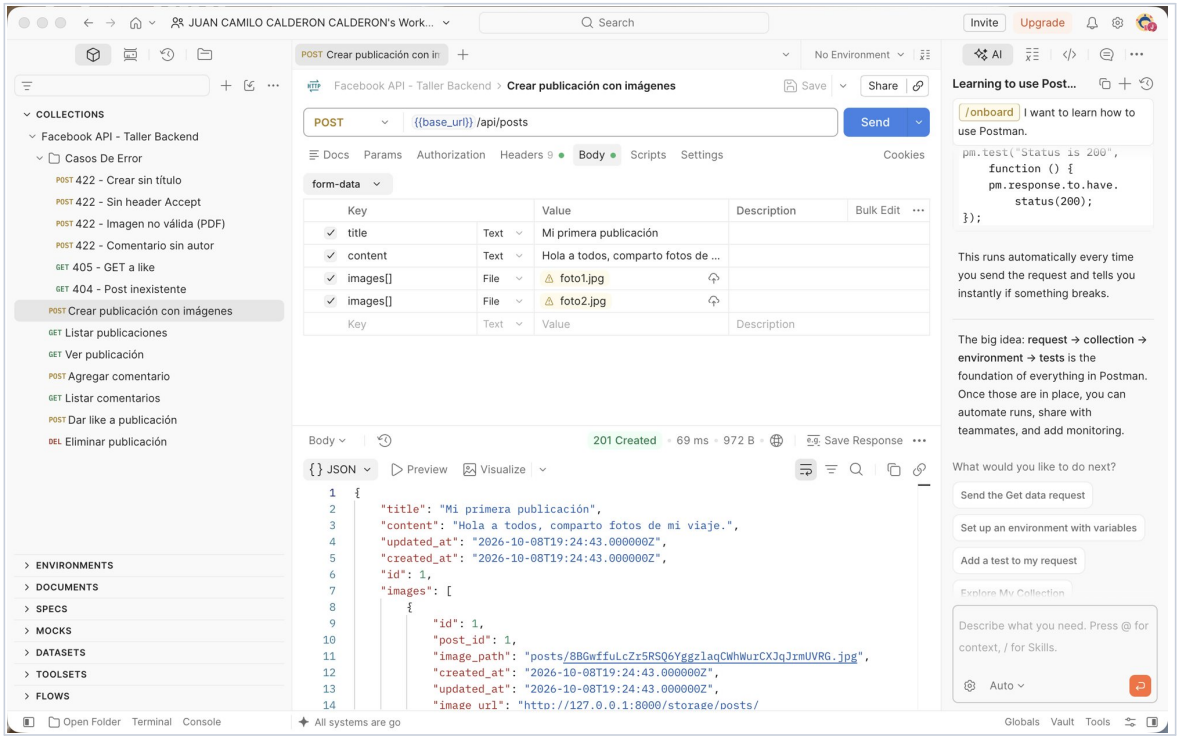


Figura 2. Postman: POST /api/posts con title, content e images[] (File) ×2. Respuesta 201 Created con el post, sus imágenes y la image_url.

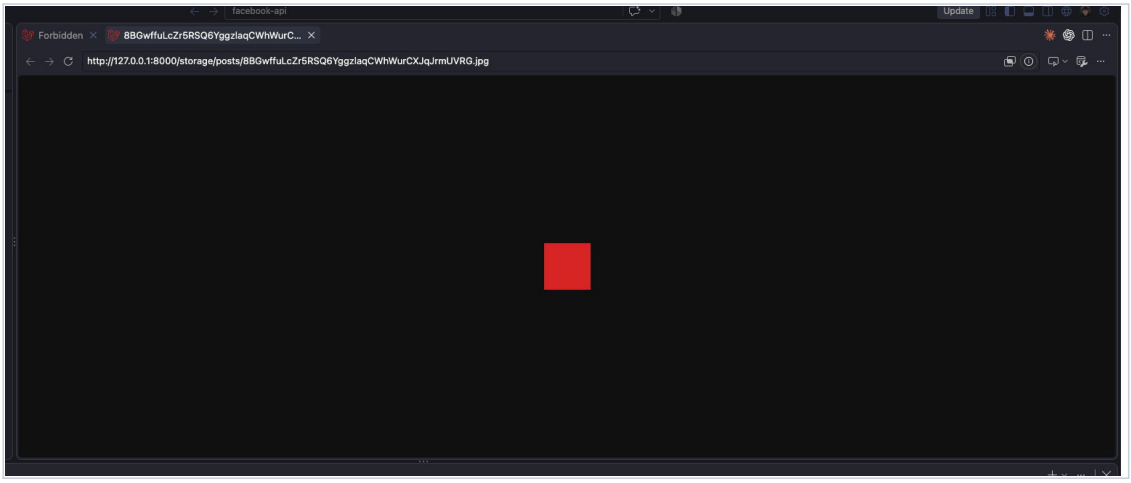


Figura 3. Navegador abriendo una image_url del post (http://127.0.0.1:8000/storage/posts/8BGwffuLcZr5RSQ6YggzlaqCWhWurCXJqJrmUVRG.jpg): la imagen se muestra, lo que prueba que storage:link funciona.

¿Por qué este método y este código? POST porque se crea un recurso nuevo; se usa multipart/form-data porque la petición lleva archivos. 201 indica que el recurso quedó creado en la base de datos.

3.2 Listar publicaciones

Petición: GET {{base_url}}/api/posts

Código esperado: 200 OK | **Código obtenido:** 200 OK

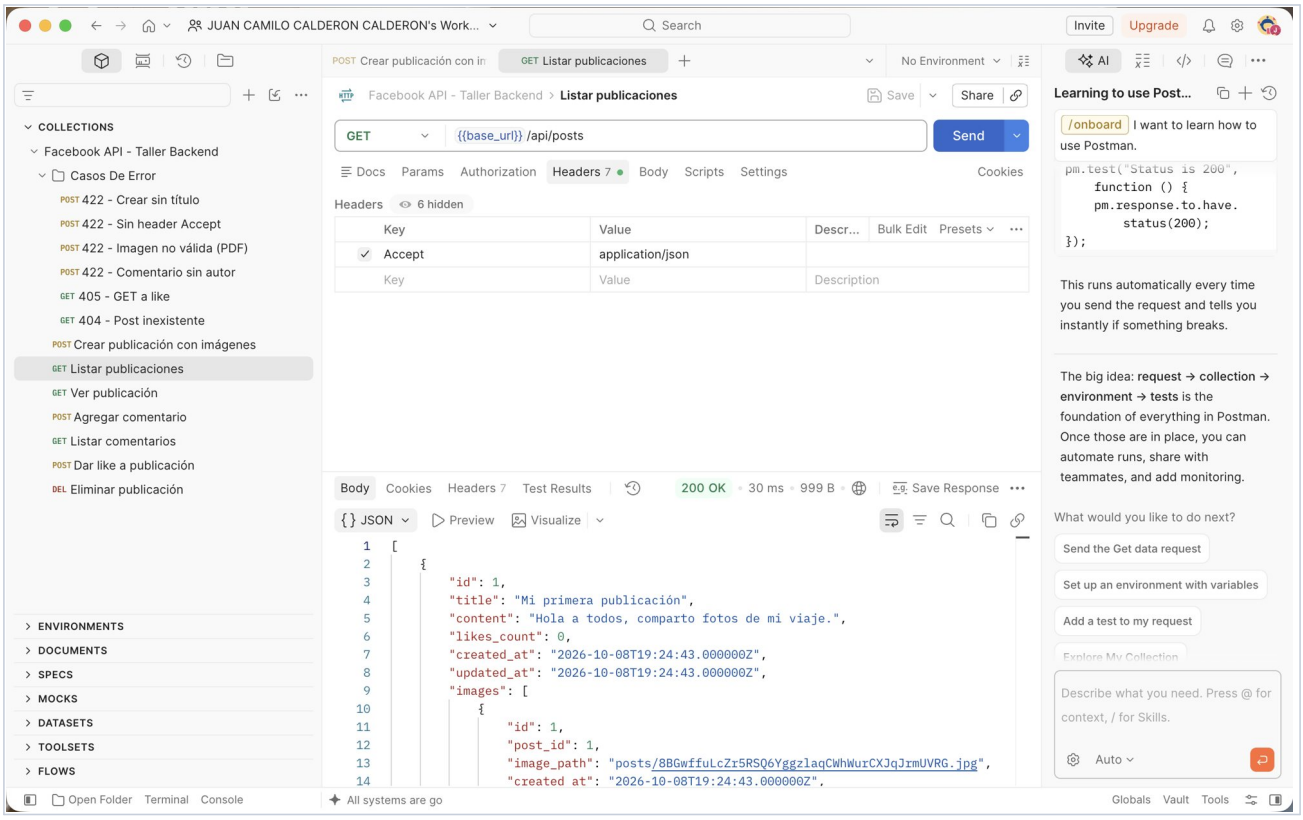


Figura 4. Postman: GET /api/posts con Accept: application/json. Respuesta 200 OK con el arreglo de posts, cada uno con images (con image_url) y comments.

¿Por qué este método y este código? GET sirve para leer datos sin modificarlos. 200 indica que la operación fue exitosa y se devuelve información.

3.3 Ver publicación

Petición: GET {{base_url}}/api/posts/1

Código esperado: 200 OK | **Código obtenido:** 200 OK

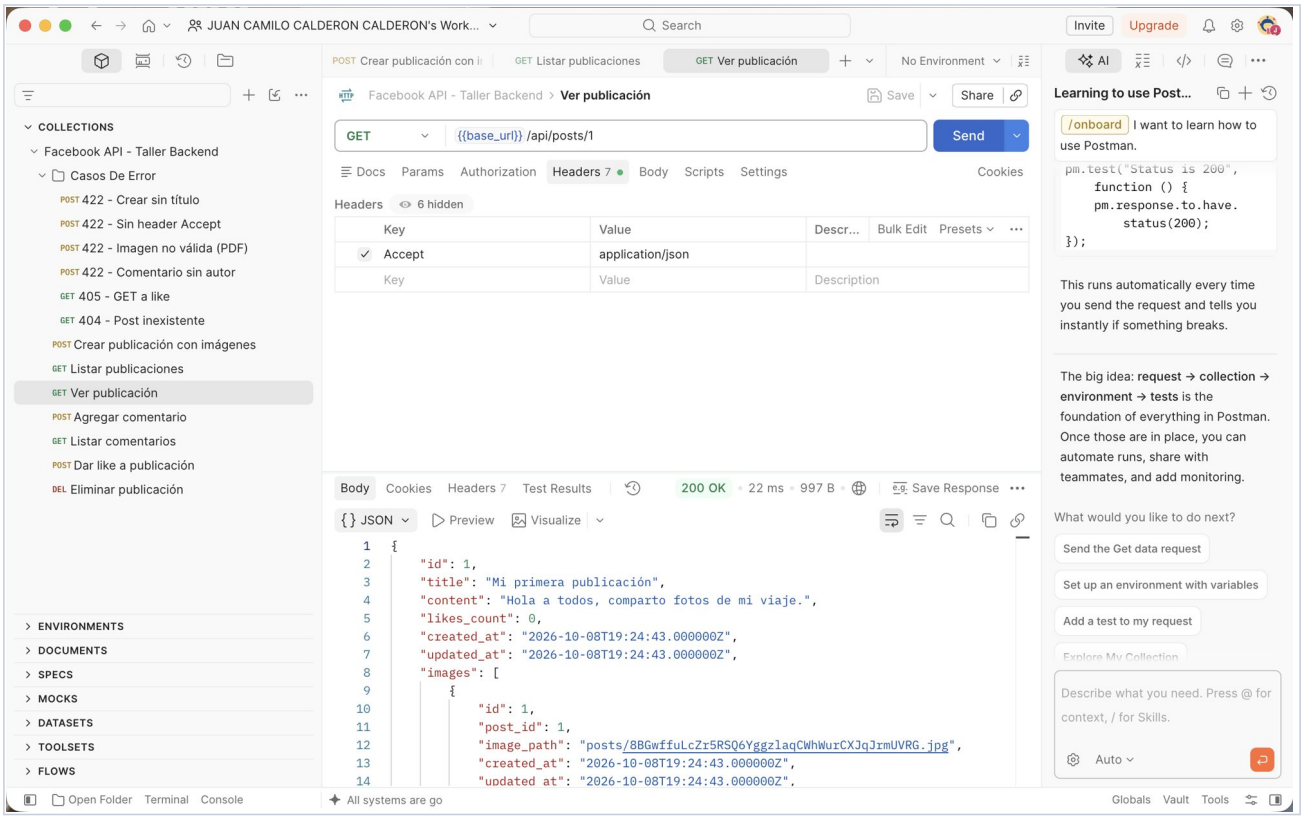


Figura 5. Postman: GET /api/posts/1. Respuesta 200 OK con el post de id 1 y sus imágenes.

¿Por qué este método y este código? GET consulta un recurso concreto identificado por el id en la URL; 200 confirma que existe y se devuelve.

3.4 Agregar comentario

Petición: POST {{base_url}}/api/posts/1/comments (raw JSON)

Código esperado: 201 Created | **Código obtenido:** 201 Created

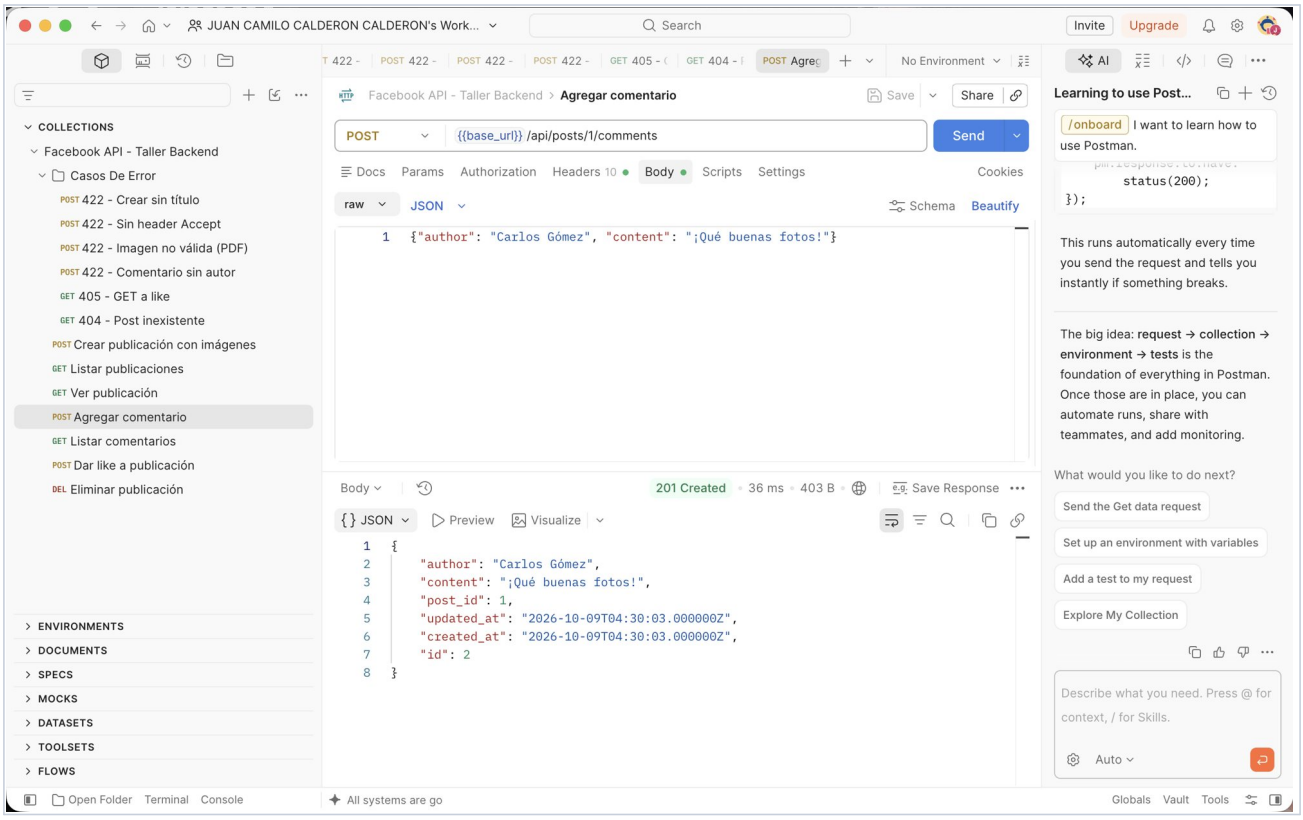


Figura 6. Postman: POST /api/posts/1/comments con {"author": "Carlos Gómez", "content": "¡Qué buenas fotos!"}. Respuesta 201 Created con el comentario creado (post_id: 1).

¿Por qué este método y este código? POST porque se crea un comentario nuevo asociado a un post existente; 201 indica que se creó correctamente.

3.5 Listar comentarios

Petición: GET {{base_url}}/api/posts/1/comments

Código esperado: 200 OK | **Código obtenido:** 200 OK

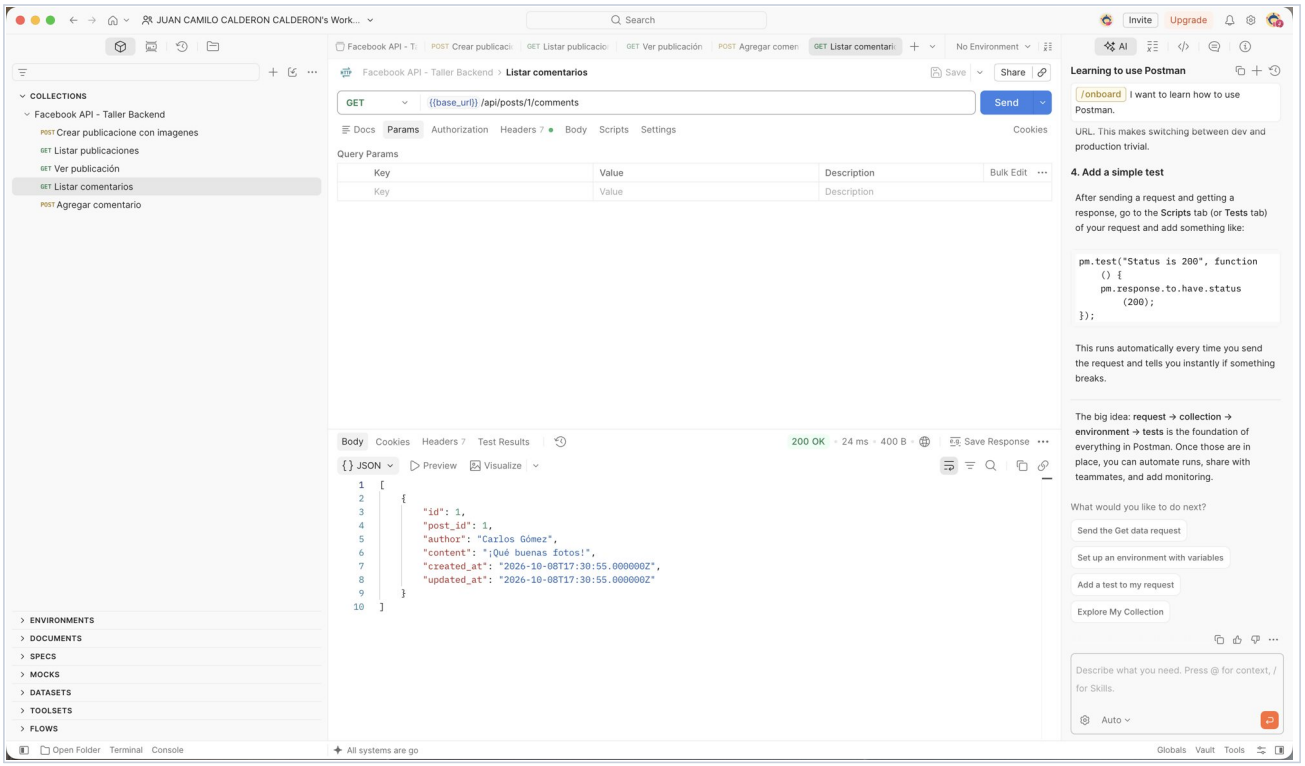


Figura 7. Postman: GET /api/posts/1/comments. Respuesta 200 OK con el comentario de "Carlos Gómez". Captura de una ejecución previa (12:32 PM) de la misma petición.

¿Por qué este método y este código? GET lee los comentarios de un post sin modificar datos; 200 indica éxito.

3.6 Dar like

Petición: POST {{base_url}}/api/posts/1/like (sin body)

Código esperado: 200 OK | **Código obtenido:** 200 OK

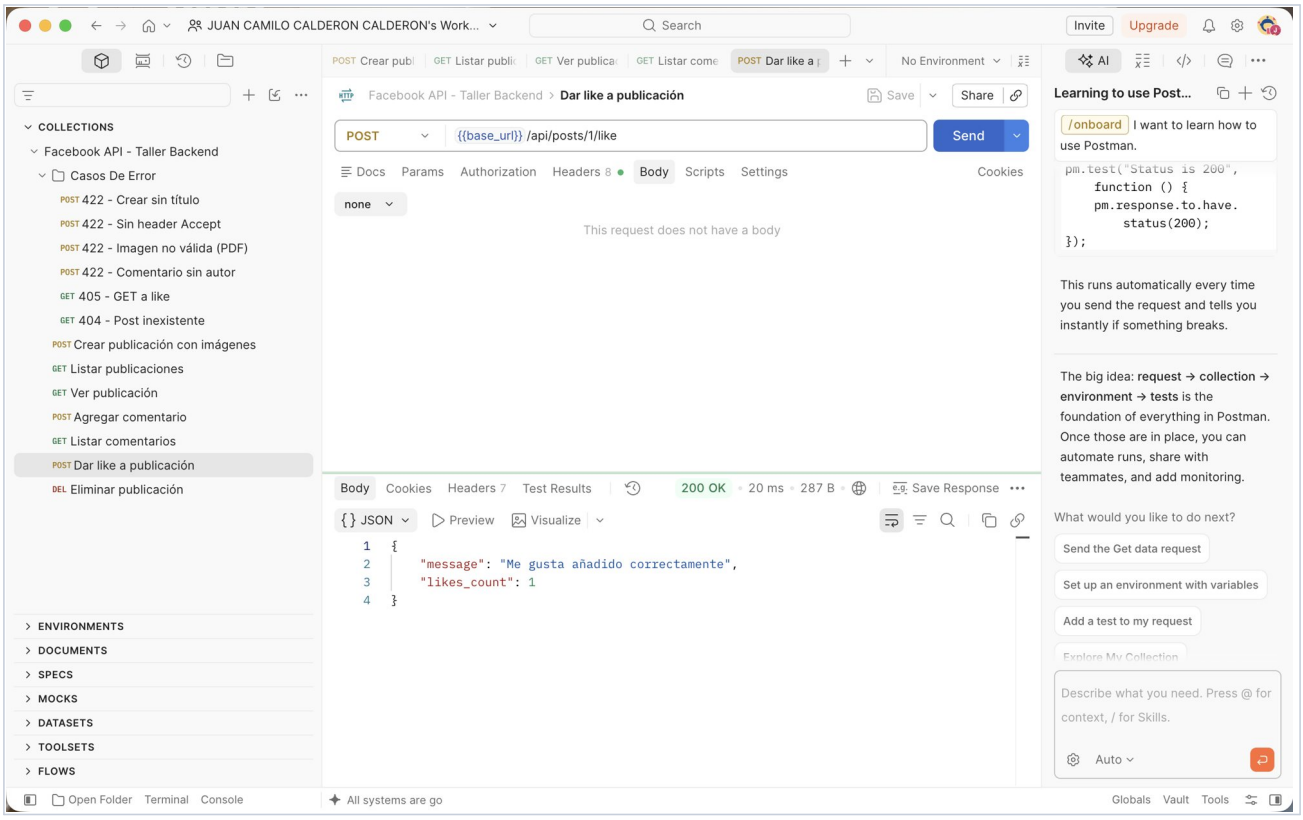


Figura 8. Postman: POST /api/posts/1/like. Respuesta 200 OK con "Me gusta añadido correctamente" y likes_count: 1.

¿Por qué este método y este código? POST porque se registra una interacción nueva; no lleva body porque el id del post va en la URL. 200 porque se devuelve información (el nuevo likes_count).

3.7 Eliminar publicación

Petición: DELETE {{base_url}}/api/posts/1

Código esperado: 204 No Content | **Código obtenido:** 204 No Content

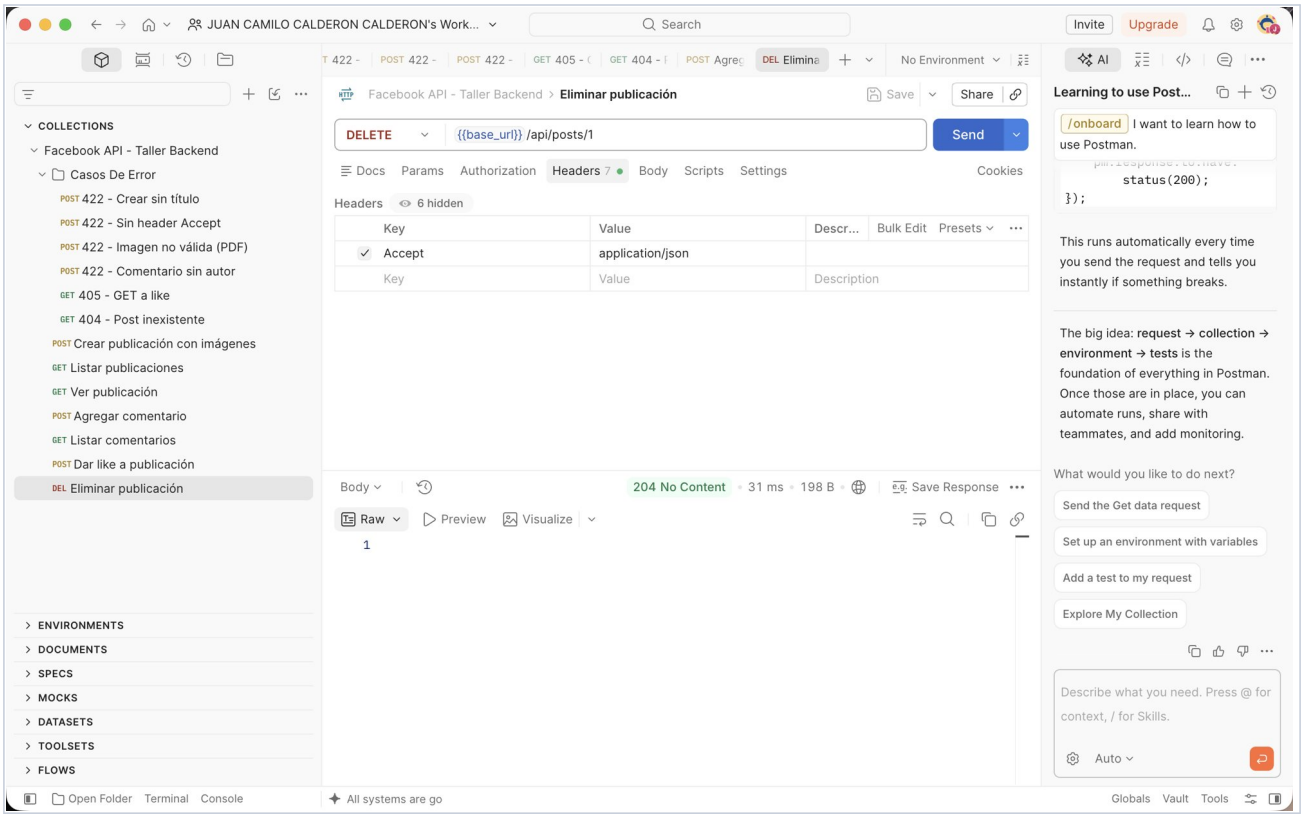


Figura 9. Postman: DELETE /api/posts/1. Respuesta 204 No Content, 198 B y cuerpo vacío.

¿Por qué este método y este código? DELETE elimina un recurso; 204 indica éxito sin cuerpo de respuesta. El borrado físico de las imágenes y el borrado en cascada se comprueban en la página siguiente.

3.7.1 Comprobación del borrado físico y en cascada

Comando ejecutado en ambos momentos, antes y después del DELETE: `ls -la storage/app/public/posts && php artisan tinker --execute="echo posts=, post_images=, comments="` (cuenta registros de las tablas posts, post_images y comments).

Antes del DELETE



Figura 10. Terminal ANTES de eliminar: se listan 2 archivos .jpg en storage/app/public/posts (fechados 14:24) y el conteo muestra posts=1 post_images=2 comments=1.

Después del DELETE



Figura 11. Terminal DESPUÉS de eliminar: la carpeta storage/app/public/posts está vacía (total 0) y el conteo muestra posts=0 post_images=0 comments=0.

Resultado: tras el DELETE desaparecen los archivos físicos de las imágenes y los registros del post, de sus imágenes y de su comentario, lo que demuestra el borrado de archivos en destroy() y el borrado en cascada definido en las claves foráneas.

3.8 Ver publicación después de eliminar

Petición: GET {{base_url}}/api/posts/1

Código esperado: 404 Not Found | **Código obtenido:** 404 Not Found

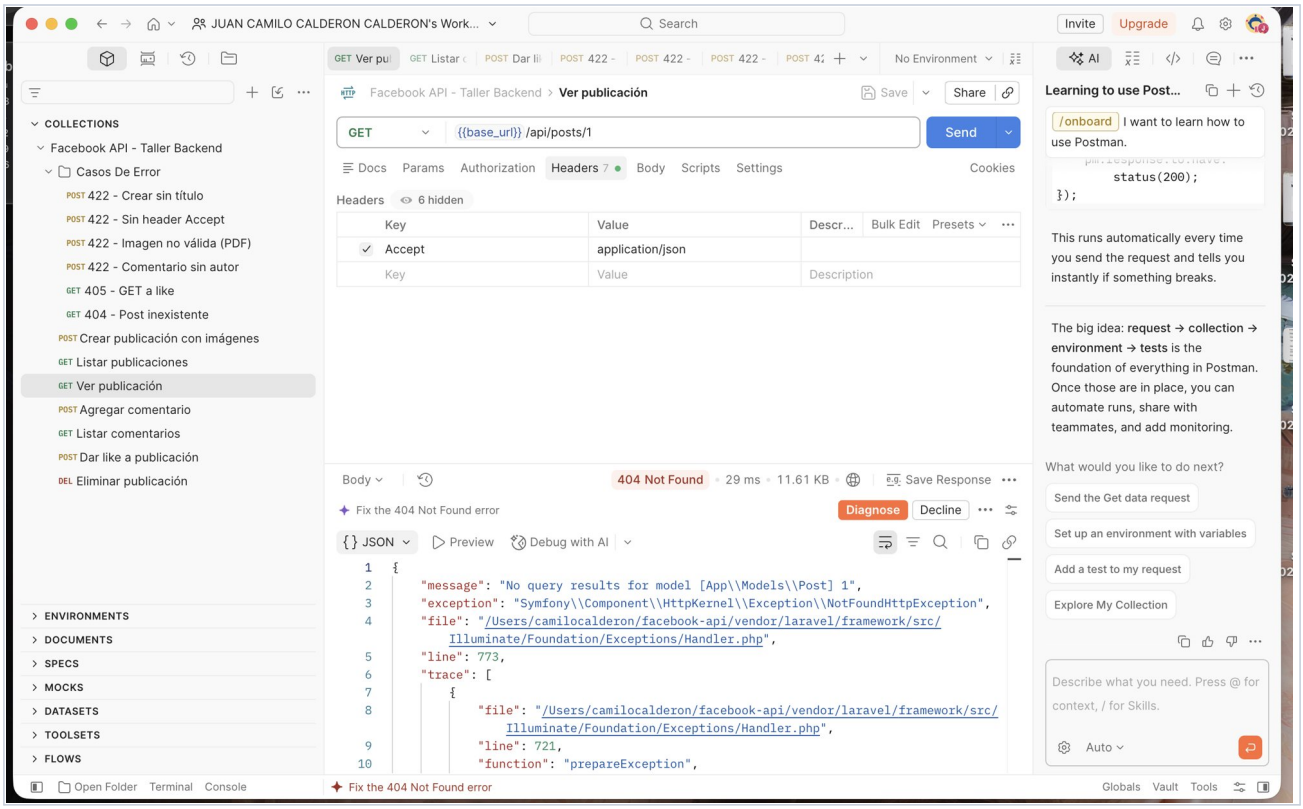


Figura 12. Postman: GET /api/posts/1 después del DELETE. Respuesta 404 Not Found: "No query results for model [App\Models\Post] 1".

¿Por qué este método y este código? 404 indica que el recurso solicitado ya no existe, lo que confirma que el post fue eliminado. La respuesta incluye rutas locales porque la aplicación está en modo debug.

4. Casos de error

Pruebas de validación y de rutas inválidas, guardadas en la carpeta "Casos De Error" de la colección **Facebook API - Taller Backend**.

4.1 422 sin título

Petición: POST {{base_url}}/api/posts (title vacío)

Código esperado: 422 | **Código obtenido:** 422 Unprocessable Entity

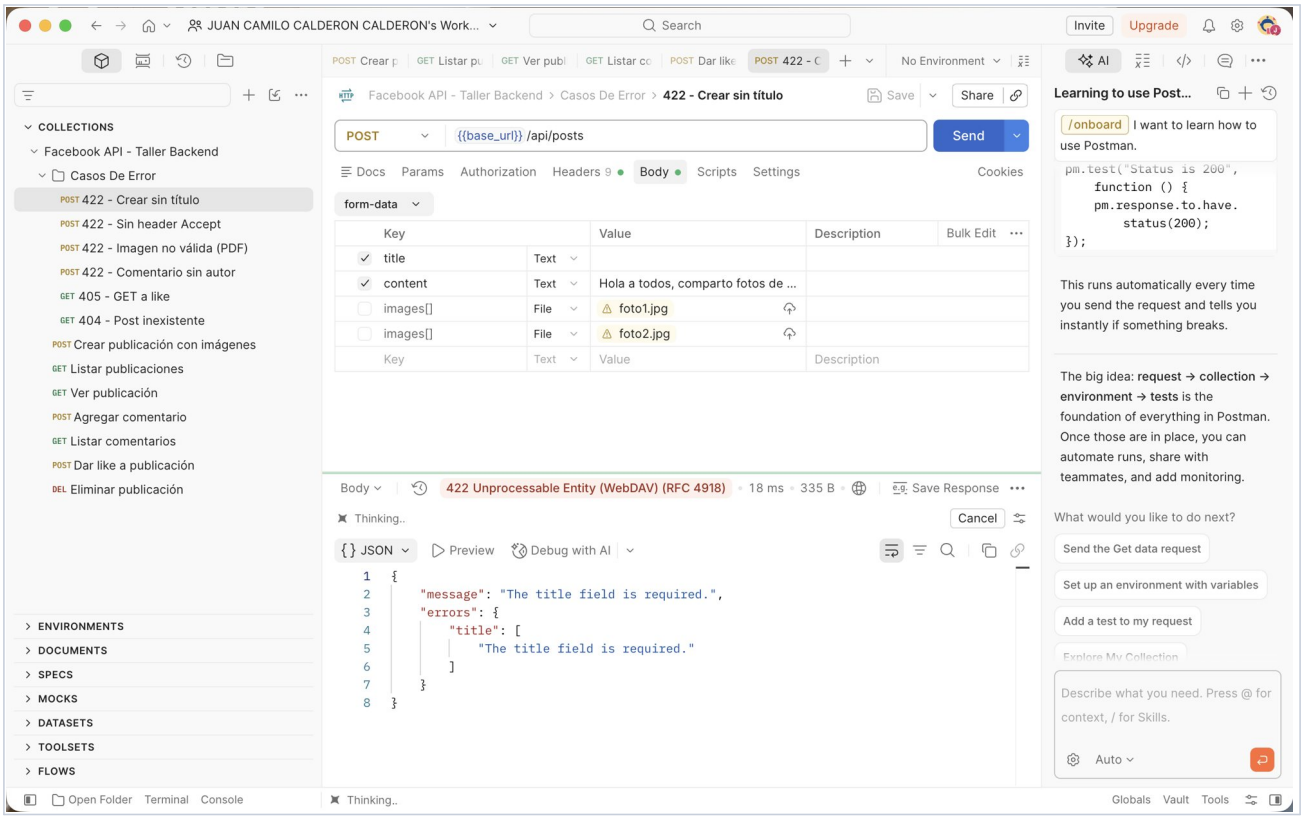


Figura 13. Postman: POST /api/posts con title vacío. Respuesta 422 con "The title field is required."

¿Por qué este método y este código? La petición llega bien pero no pasa la validación (title es obligatorio): por eso 422 y no 400.

4.2 422 con PDF como imagen

Petición: POST {{base_url}}/api/posts (images[] = documento.pdf)

Código esperado: 422 | **Código obtenido:** 422 Unprocessable Entity

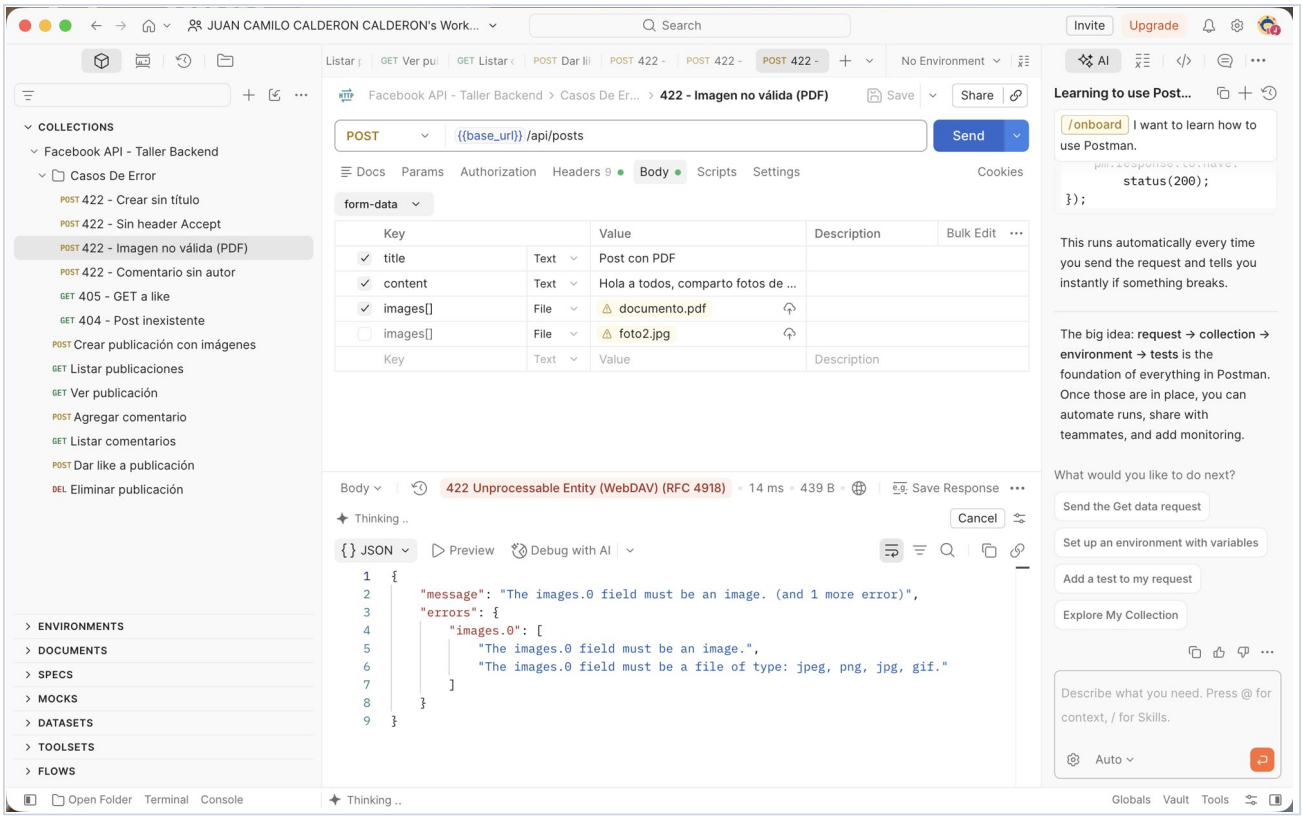


Figura 14. Postman: POST /api/posts con documento.pdf en images[]. Respuesta 422: "The images.0 field must be an image." y el tipo debe ser jpeg, png, jpg o gif.

¿Por qué este método y este código? El archivo no es una imagen, así que falla la regla images.* (image|mimes:jpeg,png,jpg,gif).

4.3 422 comentario sin autor

Petición: POST {{base_url}}/api/posts/1/comments (solo content)

Código esperado: 422 | **Código obtenido:** 422 Unprocessable Entity

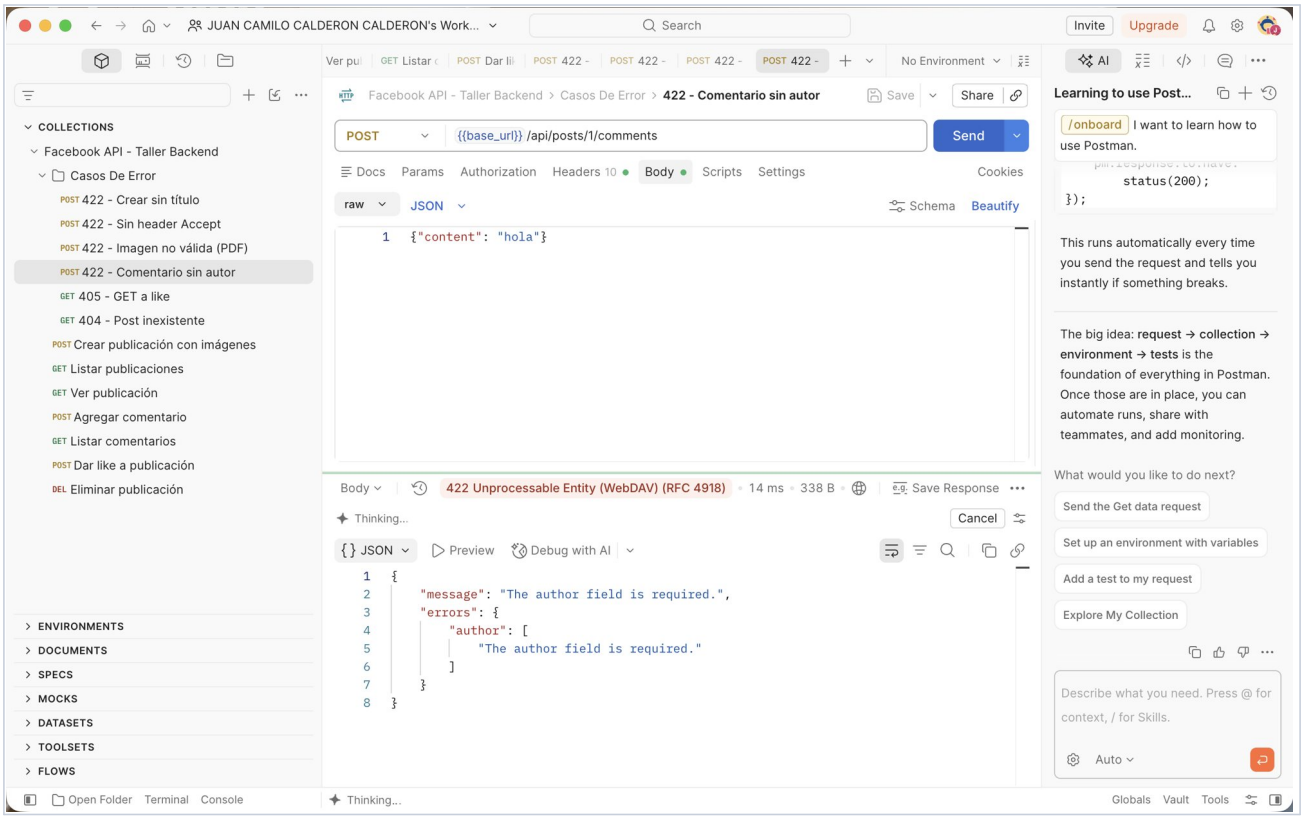


Figura 15. Postman: POST /api/posts/1/comments con {"content": "hola"}. Respuesta 422 con "The author field is required."

¿Por qué este método y este código? El campo author es obligatorio; al faltar, la validación falla y Laravel responde 422.

4.4 404 post inexistente

Petición: GET {{base_url}}/api/posts/9999

Código esperado: 404 | **Código obtenido:** 404 Not Found

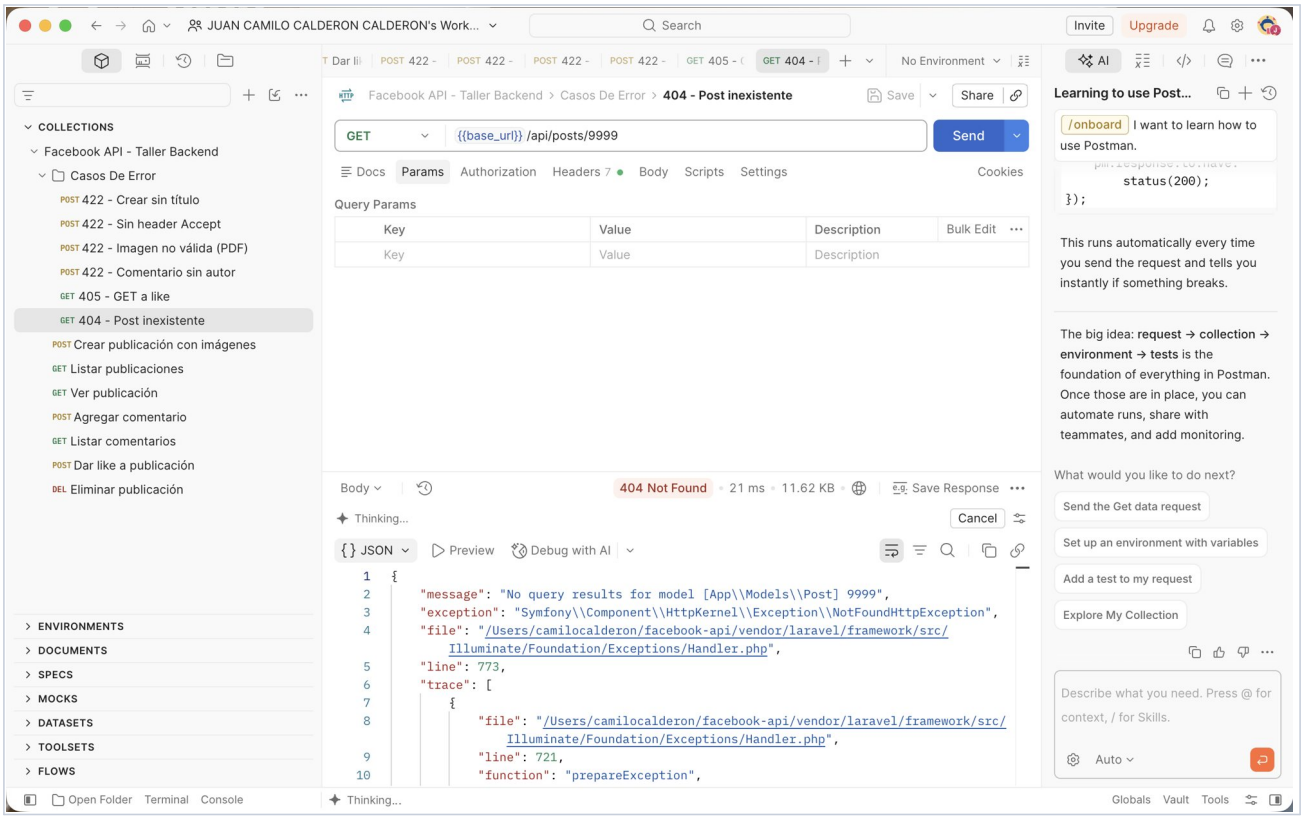


Figura 16. Postman: GET /api/posts/9999. Respuesta 404 Not Found: "No query results for model [App\Models\Post] 9999".

¿Por qué este método y este código? findOrFail no encuentra el post 9999 y Laravel responde 404: el recurso no existe. La respuesta incluye rutas locales por estar en modo debug.

4.5 405 GET a /api/posts/1/like

Petición: GET {{base_url}}/api/posts/1/like

Código esperado: 405 | **Código obtenido:** 405 Method Not Allowed

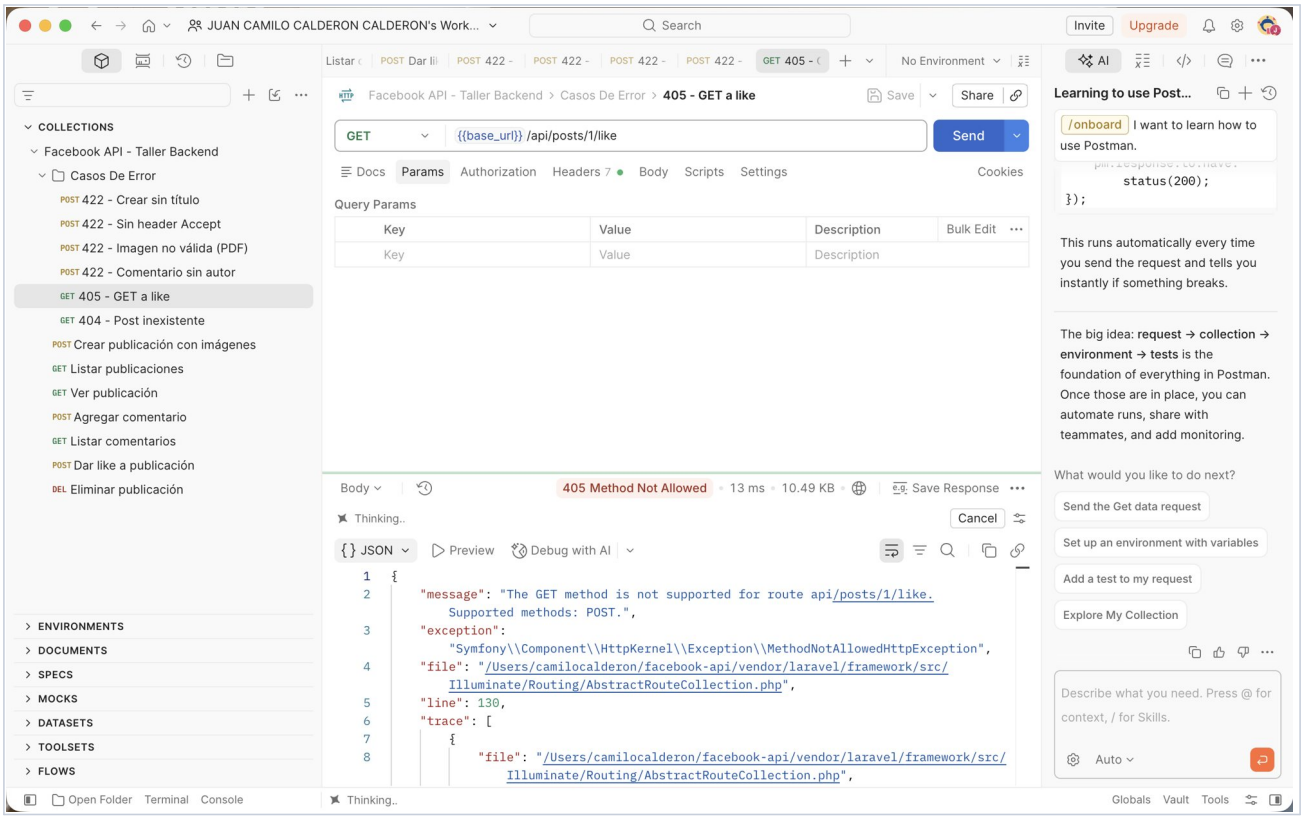


Figura 17. Postman: GET /api/posts/1/like. Respuesta 405 Method Not Allowed: "The GET method is not supported for route api/posts/1/like. Supported methods: POST."

¿Por qué este método y este código? La ruta existe pero solo acepta POST; usar otro verbo devuelve 405.

4.6 422 sin header Accept

Petición: POST {{base_url}}/api/posts (title vacío, sin Accept: application/json)

Código esperado: 422 en JSON | **Código obtenido:** 422 Unprocessable Entity (JSON)

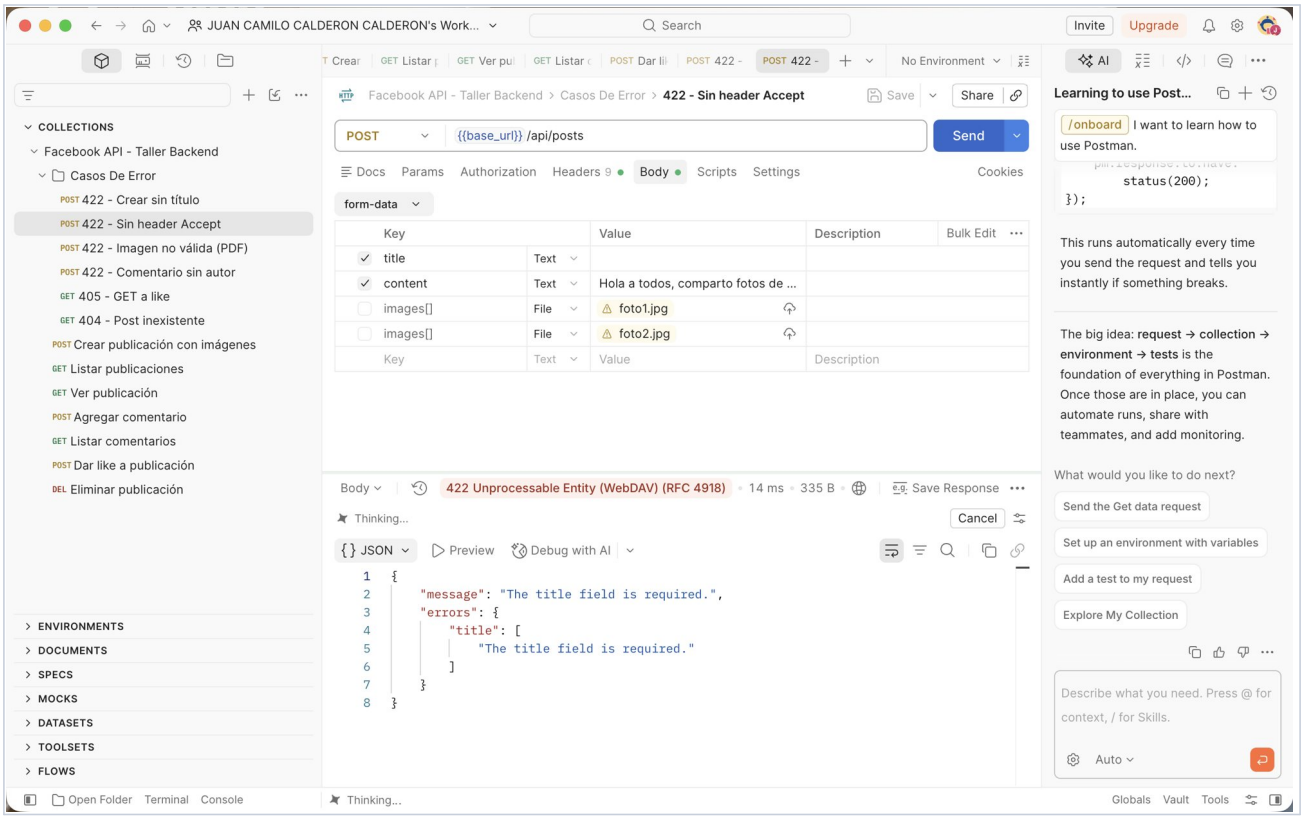


Figura 18. Postman: petición "422 - Sin header Accept" a POST /api/posts con title vacío. Respuesta 422 en JSON: "The title field is required."

¿Por qué este método y este código? Sin el header Accept, en Laravel 10/11 un error de validación se trataba como petición web y redirigía con 302 a la página anterior. En Laravel 13 la API responde JSON igual, porque bootstrap/app.php tiene shouldRenderJsonWhen para las rutas api/*, que fuerza la respuesta JSON en esas rutas.

5. Repositorio y control de versiones

Repositorio: https://github.com/CamiloCalder0n/-facebook-api

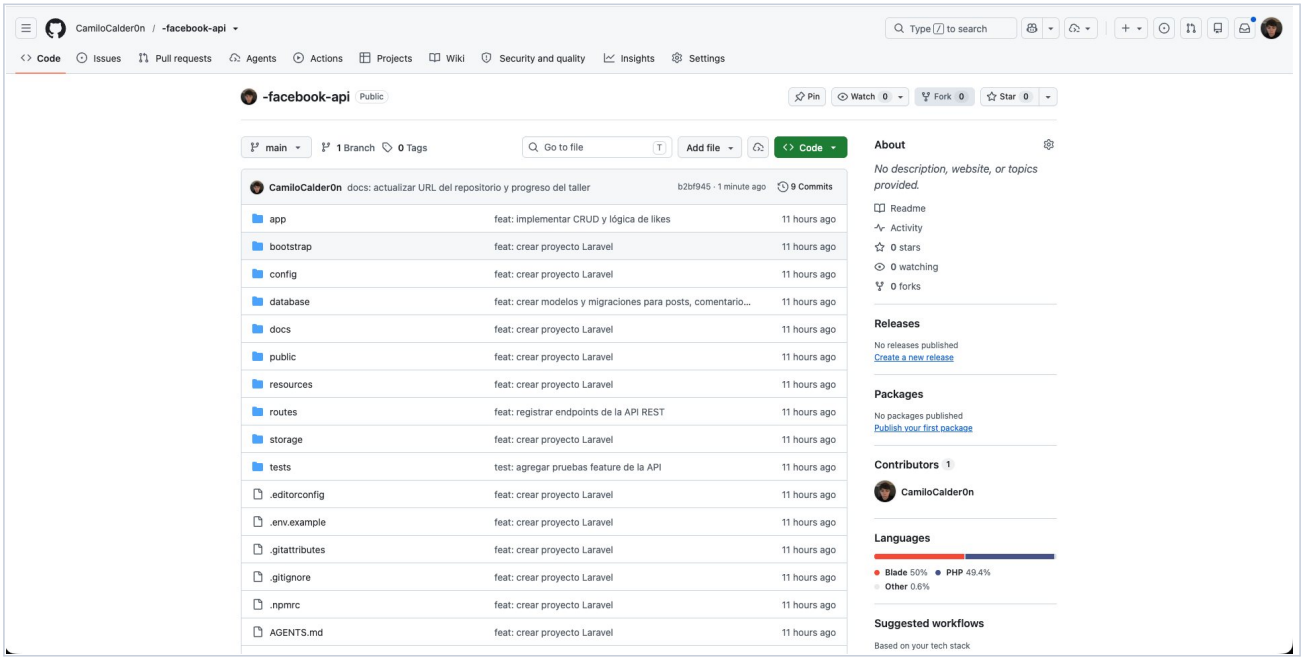


Figura 19. GitHub: repositorio público "-facebook-api" (rama main, 9 commits). Último commit visible: "docs: actualizar URL del repositorio y progreso del taller" (b2bf945). En la columna "About" aparece el enlace Readme.

Historial de commits

Commits registrados en la terminal (más reciente primero), con el formato Conventional Commits que pide el taller:

Commit	Mensaje (Conventional Commits)	Tipo
27a86a0	test: exportar colección de Postman	test
e321cbd	test: agregar pruebas feature de la API	test
87bd643	feat: registrar endpoints de la API REST	feat
16c8ef6	feat: implementar CRUD y lógica de likes	feat
7ba2143	feat: configurar relaciones entre Post, Comment y PostImage	feat
e5720fa	feat: crear modelos y migraciones para posts, comentarios e imágenes	feat
18575cd	feat: crear proyecto Laravel	feat

El archivo de terminal se tomó a las 14:21 y registra 7 commits; la captura de GitHub, tomada más tarde, muestra 9 en total (los 2 posteriores no figuran en ese archivo).

Archivos sensibles fuera del repositorio

```
$ git ls-files | grep -E '\.env$|sqlite'   (vacío = no se versionan)
(vacío)

$ git ls-files facebook-api-postman-collection.json README.md
README.md
facebook-api-postman-collection.json
```

6. Checklist "Dejarlo funcionando"

Verificación	Figura / evidencia	Estado
Proyecto corriendo en http://127.0.0.1:8000	Figura 1 y "curl -I" con HTTP 200 (sección 2)	**Cumplido**
Enlace simbólico de storage creado	"ls -la public/storage" (sección 2) y Figura 3	**Cumplido**
Tablas posts, post_images y comments migradas	"php artisan migrate:status" (sección 2)	**Cumplido**
POST /api/posts crea el post y guarda las imágenes (201)	Figura 2, Figura 3 y Figura 10	**Cumplido**
GET /api/posts lista con image_url completo	Figura 4	**Cumplido**
POST /api/posts/{id}/comments agrega comentario (201)	Figura 6	**Cumplido**
POST /api/posts/{id}/like incrementa likes_count (200)	Figura 8	**Cumplido**
DELETE /api/posts/{id} elimina el post y las imágenes físicas (204)	Figuras 9 (204), 10 (antes) y 11 (después)	**Cumplido**
Colección de Postman completa y exportada	Figuras 2 a 18 y "git ls-files" con facebook-api-postman-collection.json (sección 5)	**Cumplido**
Repositorio en GitHub con README.md	Figura 19 y "git ls-files" con README.md (sección 5)	**Cumplido**

Pruebas automáticas adicionales: 15 de 15 en verde (88 aserciones).

Universidad Autónoma de Bucaramanga · UNAB

Evidencias del taller · Parte 2

Retos Avanzados de API REST con Laravel

Continuación de la API REST tipo Facebook

Autor	Juan Camilo Calderon (trabajo individual)
Materia	Arquitectura y Desarrollo Backend
Institución	Universidad Autónoma de Bucaramanga - UNAB
Docente	Fabian Enrique Suarez Carvajal
Fecha	12 de octubre de 2026
Repositorio	https://github.com/CamiloCalder0n/-facebook-api (la Parte 1 queda en el tag `parte-1`)
Stack	Laravel 13 · PHP 8.4 · SQLite · Laravel Sanctum · Scribe · Mailtrap (sandbox) · Postman / Newman

1. Resumen

La Parte 2 extiende la API de publicaciones con los 7 retos del taller. Los códigos HTTP se toman principalmente de la corrida de la colección de Postman por terminal con Newman (archivo `01-newman-resultado.txt`): 25 requests de la colección, 36 peticiones HTTP en total y 65 de 65 aserciones aprobadas. Los bloques de código de este informe son extractos literales de los archivos de evidencia; solo se omiten las advertencias de Node que imprime Newman.

Reto	Cómo se resolvió	Evidencia
1 · Autenticación con Sanctum	Registro, login, logout y GET /user con tokens Bearer; rutas de escritura protegidas con auth:sanctum; user_id en posts y comentarios; solo el dueño puede borrar (403).	Figuras 1 y 2, Newman
2 · Paginación	GET /posts con paginate(10) e información de paginación.	Newman
3 · Likes por usuario	Tabla pivote post_user (User ↔ Post muchos a muchos); un like repetido responde 409.	Figura 3, Newman
4 · Búsqueda	Parámetro ?search= que filtra por título o contenido.	Newman
5 · Correo al comentar	Mailable "Nuevo comentario" enviado al dueño del post; probado en Mailtrap.	Figura 4, Newman
6 · Versionado	/api/v1, alias /api sin versión y /api/v2 con controlador y formato propios.	Newman, route:list
7 · Documentación	Documentación generada con Scribe en /docs.	Figura 5

2. Retos

Reto 1 · Autenticación con Laravel Sanctum

Petición	Esperado	Obtenido
POST /api/v1/register (usuarios A y B)	201 + token	201 Created
POST /api/v1/login (A y B)	200 + token	200 OK
POST /api/v1/login con clave incorrecta	401	401 Unauthorized
GET /api/v1/user con token	200	200 OK
POST /api/v1/posts sin token	401	401 Unauthorized
DELETE /api/v1/posts/1 por un usuario que no es el dueño	403	403 Forbidden
POST /api/v1/logout y luego **GET** /api/v1/user con el mismo token	200 → 401	200 OK → 401

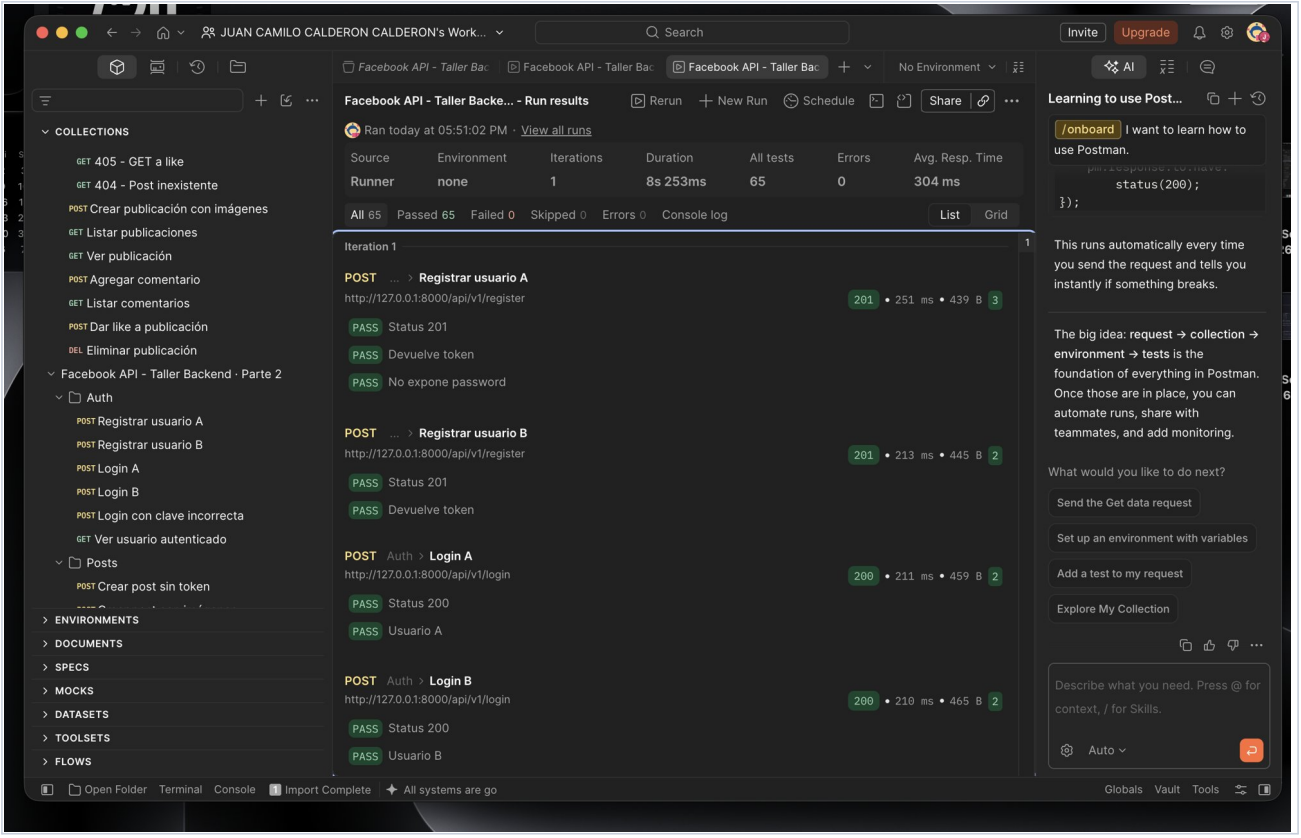


Figura 1. Collection Runner de Postman (corrida de las 5:51 PM): 65 pruebas, 65 aprobadas y 0 fallidas. Registrar usuario A y B → 201 ("Devuelve token", "No expone password"); Login A y B → 200.

Autenticación y rutas protegidas · fuente: 01-newman-resultado.txt

```
↳ Login con clave incorrecta
  POST http://127.0.0.1:8000/api/v1/login [401 Unauthorized, 271B, 210ms]
  ✓  Status 401
  ✓  Mensaje

↳ Ver usuario autenticado
  GET http://127.0.0.1:8000/api/v1/user [200 OK, 389B, 5ms]
  ✓  Status 200
  ✓  Es el usuario A

↳ Crear post sin token
  POST http://127.0.0.1:8000/api/v1/posts [401 Unauthorized, 260B, 4ms]
  ✓  Status 401
  ✓  Unauthenticated

↳ Crear post con imágenes
  POST http://127.0.0.1:8000/api/v1/posts [201 Created, 998B, 11ms]
  ✓  Status 201
  ✓  Guarda user_id del autenticado
  ✓  2 imágenes con image_url
```

Migraciones nuevas para asociar el usuario · fuente: 02-evidencia-terminal.txt

```
2026_10_09_050232_add_user_id_to_posts_table .. [1] Ran
2026_10_09_050233_add_user_id_to_comments_table .. [1] Ran
```

¿Por qué? Sanctum emite un token por usuario y el middleware `auth:sanctum` lo valida en cada petición: sin token, o con un token revocado, la respuesta es 401 ("no sé quién eres"). Si el usuario está identificado pero no es el dueño del recurso, la respuesta es 403 ("sé quién eres, pero no tienes permiso"). La autorización de borrado y el logout se ven en la Figura 6 (sección 3).

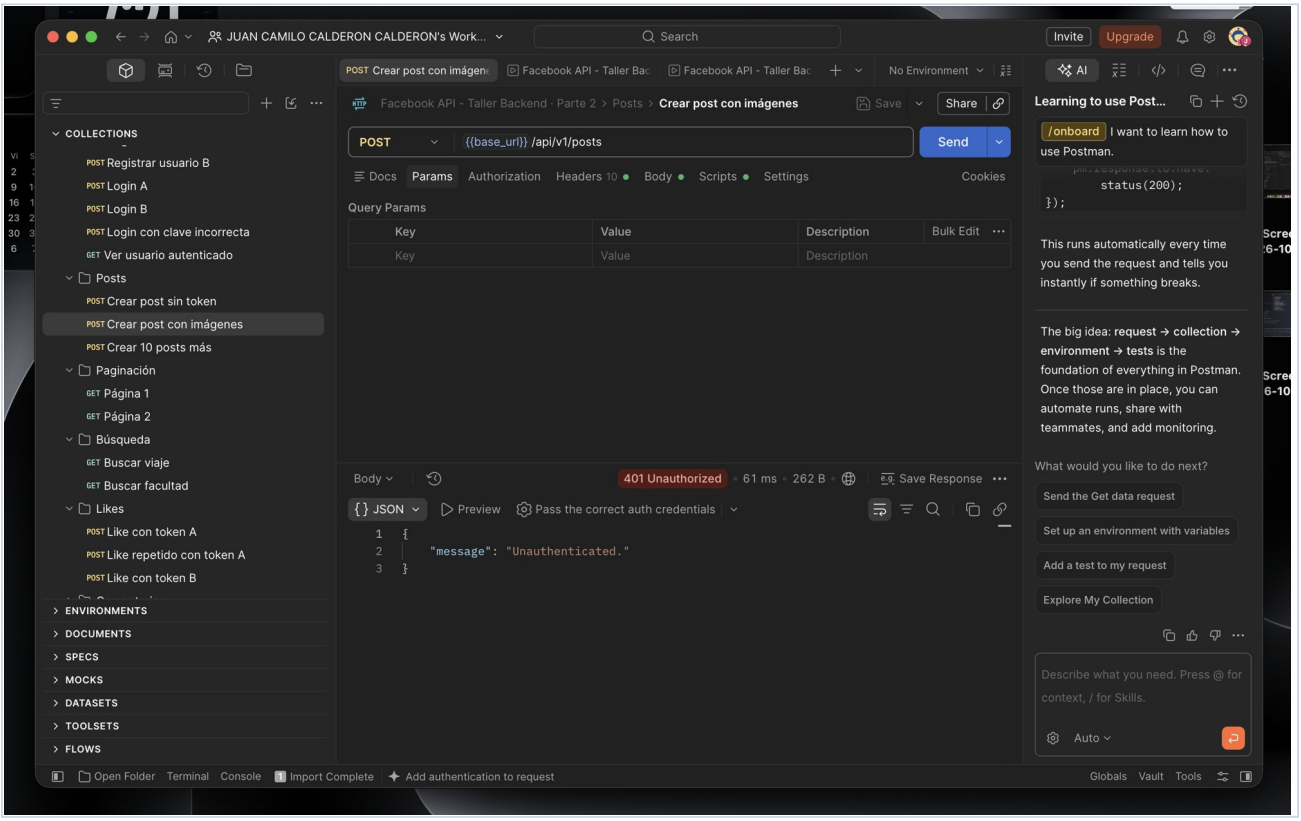


Figura 2. POST /api/v1/posts ("Crear post con imágenes") enviado después del logout del usuario A, con su token ya revocado: 401 Unauthorized y {"message": "Unauthenticated."}.

¿Por qué? El logout borra el token del usuario en la base de datos (`personal_access_tokens`), así que ese mismo token deja de servir aunque el cliente lo siga enviando.

Reto 2 · Paginación de publicaciones

Petición	Esperado	Obtenido
GET /api/v1/posts?page=1	200, 10 elementos	200 OK, 10 elementos
GET /api/v1/posts?page=2	200, el resto	200 OK, 1 elemento

Paginación · fuente: 01-newman-resultado.txt

```
↳ Crear 10 posts más
  POST http://127.0.0.1:8000/api/v1/posts [201 Created, 434B, 6ms]
  ✓  Status 201
  ✓  Se crearon los 10 posts extra

↳ Página 1
  GET http://127.0.0.1:8000/api/v1/posts?page=1 [200 OK, 3.89kB, 7ms]
  ✓  Status 200
  ✓  10 elementos
  ✓  Info de paginación

↳ Página 2
  GET http://127.0.0.1:8000/api/v1/posts?page=2 [200 OK, 1.18kB, 5ms]
  ✓  Status 200
  ✓  El resto (1 elemento)
  ✓  current_page 2
```

Antes de paginar se crearon 11 publicaciones (1 con imágenes y 10 extra): la página 1 trae 10 y la página 2 trae la restante. Newman verifica que la respuesta incluye la información de paginación, aunque no imprime su valor de `total`.

¿Por qué? `paginate(10)` devuelve bloques de 10 registros junto con los metadatos (página actual, total, enlaces), así el cliente no descarga todo de una vez y puede pedir la página siguiente.

Reto 4 · Búsqueda por título o contenido

Petición	Esperado	Obtenido
GET /api/v1/posts?search=viaje	200, solo coincidencias	200 OK
GET /api/v1/posts?search=facultad	200, coincidencia por contenido	200 OK

Búsqueda · fuente: 01-newman-resultado.txt

```
↳ Buscar viaje
  GET http://127.0.0.1:8000/api/v1/posts?search=viaje [200 OK, 1.6kB, 5ms]
  ✓  Status 200
  ✓  Solo coincidencias

↳ Buscar facultad
  GET http://127.0.0.1:8000/api/v1/posts?search=facultad [200 OK, 1.06kB, 5ms]
  ✓  Status 200
  ✓  Encuentra por contenido
```

¿Por qué? El filtro se aplica en SQL (`where title like … or content like …`), así la base de datos devuelve solo las filas que coinciden y la búsqueda se combina con la paginación. Buscar no cambia datos, por eso es un GET con un parámetro de consulta.

Reto 3 · Likes únicos por usuario (tabla pivote post_user)

Petición	Esperado	Obtenido
POST /api/v1/posts/1/like con token A	200, likes_count 1	200 OK, likes_count 1
POST /api/v1/posts/1/like otra vez con token A	409	409 Conflict, conteo sin cambio
POST /api/v1/posts/1/like con token B	200, likes_count 2	200 OK, likes_count 2

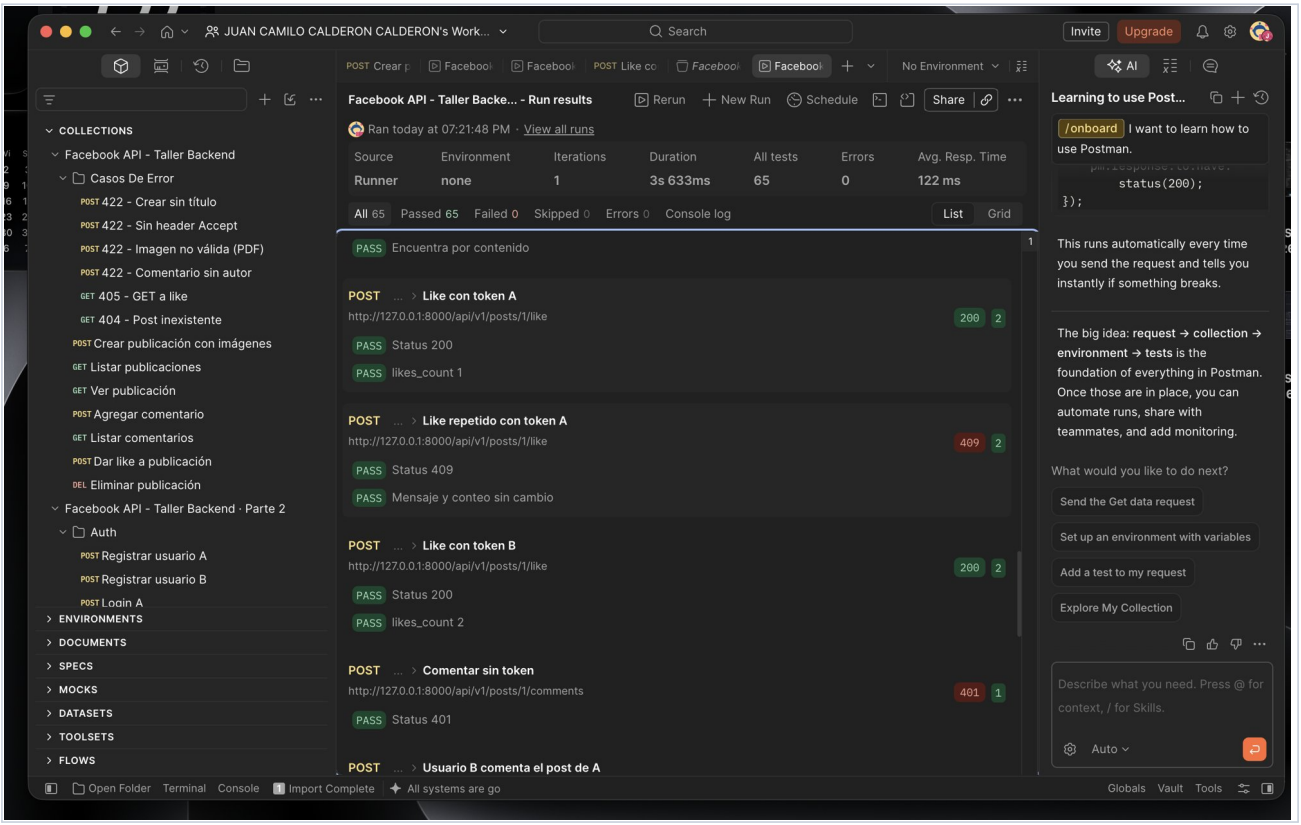


Figura 3. Collection Runner (corrida de las 7:21 PM, 65/65): Like con token A → 200 y likes_count 1; Like repetido con token A → 409 ("Mensaje y conteo sin cambio"); Like con token B → 200 y likes_count 2; Comentar sin token → 401.

Migración de la tabla pivote · fuente: 02-evidencia-terminal.txt

```
2026_10_09_050353_create_post_user_table .. [1] Ran
```

¿Por qué? La tabla `post_user` guarda qué usuario dio like a qué post (relación muchos a muchos), por lo que el conteo es real y no un contador global. Repetir el like choca con un estado ya existente, y eso es justo lo que significa 409 Conflict.

Reto 5 · Correo al dueño del post al comentar

Petición	Esperado	Obtenido
POST /api/v1/posts/1/comments sin token	401	401 Unauthorized
POST /api/v1/posts/1/comments (usuario B en el post de A)	201 + correo al dueño	201 Created
GET /api/v1/posts/1/comments	200	200 OK, 1 comentario

Comentarios · fuente: 01-newman-resultado.txt

```
↳ Usuario B comenta el post de A
  POST http://127.0.0.1:8000/api/v1/posts/1/comments [201 Created, 614B, 1532ms]
  ✓  Status 201
  ✓  author es el usuario B

↳ Listar comentarios
  GET http://127.0.0.1:8000/api/v1/posts/1/comments [200 OK, 410B, 13ms]
  ✓  Status 200
  ✓  1 comentario
```

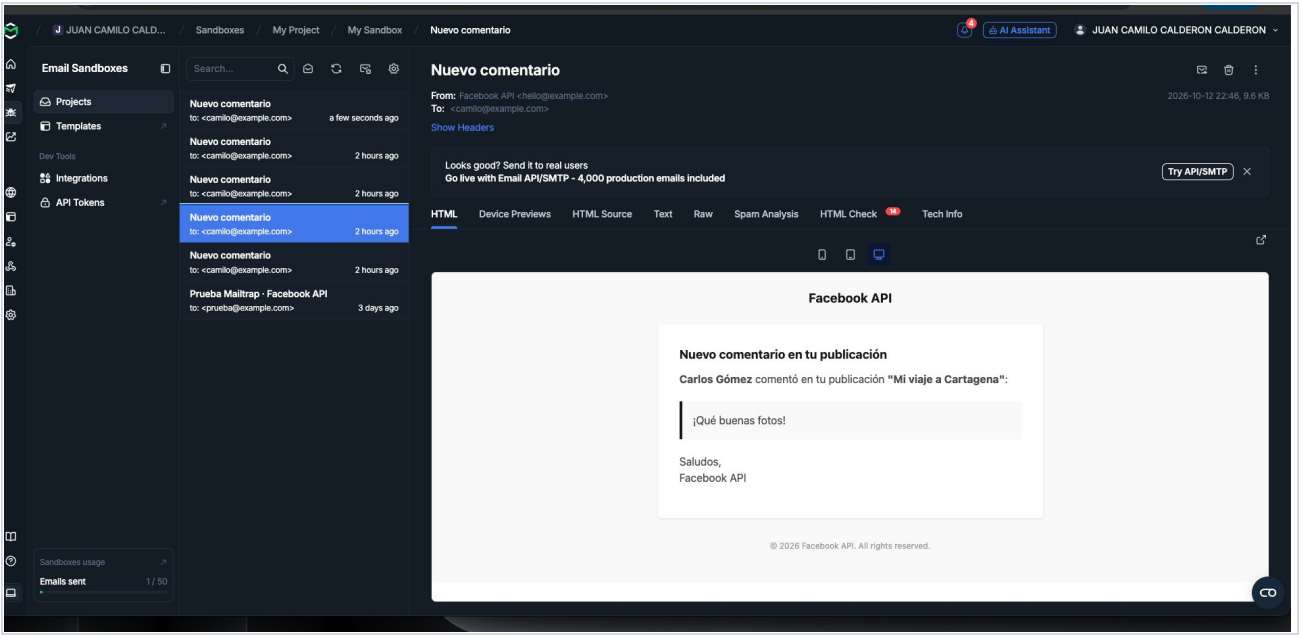


Figura 4. Mailtrap (sandbox): correo con asunto "Nuevo comentario" enviado por Facebook API: "Carlos Gómez comentó en tu publicación 'Mi viaje a Cartagena'", con el texto "¡Qué buenas fotos!".

¿Por qué? El comentario se crea con POST (201) y, al guardarse, el Mailable avisa al dueño del post sin que nadie lo haga manualmente. Mailtrap atrapa el correo para probarlo sin enviarlo a una bandeja real.

Reto 6 · Versionado de la API

Petición	Esperado	Obtenido
GET /api/posts (alias sin versión) y **GET** /api/v1/posts	200, mismo formato	200 OK en ambas, mismo formato y datos
GET /api/v2/posts	200, formato nuevo	200 OK, author, images, comments_count, links y meta

v1, alias y v2 · fuente: 01-newman-resultado.txt

```
↳ v1 vs alias /api/posts
  GET http://127.0.0.1:8000/api/posts [200 OK, 4.05kB, 10ms]
  ✓  Status 200
  GET http://127.0.0.1:8000/api/v1/posts [200 OK, 4.08kB, 9ms]
  ✓  v1 responde 200
  ✓  Mismo formato y datos

↳ v2 posts (formato nuevo)
  GET http://127.0.0.1:8000/api/v2/posts [200 OK, 3.49kB, 7ms]
  ✓  Status 200
  ✓  author como objeto
  ✓  images {id, url}
  ✓  comments_count en vez de comments
  ✓  links y meta
```

Rutas registradas (extracto) · fuente: 02-evidencia-terminal.txt

```
$ php artisan route:list --path=api
  GET|HEAD api/posts .. Api\V1\PostController@index
  GET|HEAD api/v1/posts .. Api\V1\PostController@index
  GET|HEAD api/v2/posts .. Api\V2\PostController@index
  GET|HEAD api/v2/posts/{id} .. Api\V2\PostController@show

  Showing [25] routes
```

El listado completo del archivo tiene 25 rutas: 11 con prefijo /api/v1, las mismas 11 sin versión (/api, servidas por los controladores V1), 2 de /api/v2 (controlador V2) y 1 de Scribe.

¿Por qué? Versionar permite introducir un formato nuevo (v2, con su propio controlador) sin romper a los clientes que ya usan v1. Las rutas sin versión siguen respondiendo igual que v1 para no afectar a quien consumía la API de la Parte 1.

Reto 7 · Documentación automática con Scribe

Petición: GET http://127.0.0.1:8000/docs | **Resultado:** página de documentación generada.

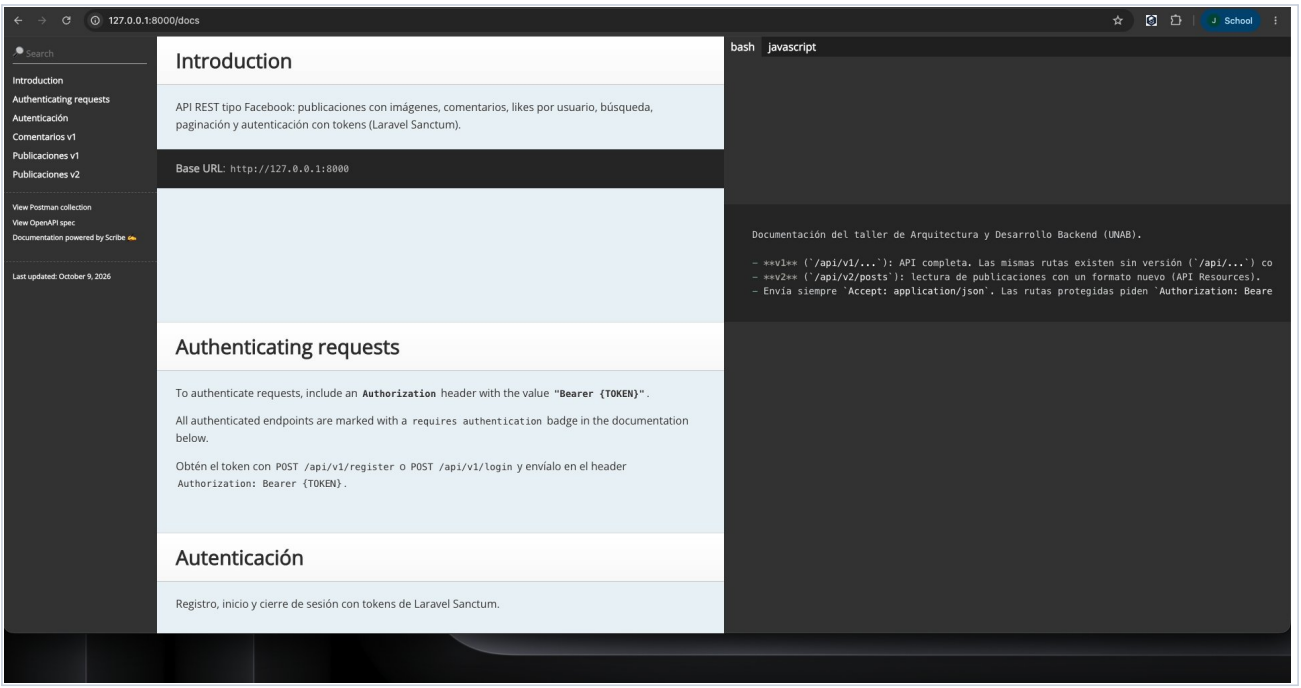


Figura 5. Documentación generada con Scribe en http://127.0.0.1:8000/docs: introducción, autenticación con "Authorization: Bearer {TOKEN}" y secciones Autenticación, Comentarios v1, Publicaciones v1 y Publicaciones v2, con enlaces a la colección de Postman y a la especificación OpenAPI.

¿Por qué? Scribe genera la documentación a partir de las rutas, las reglas de validación y los comentarios del código, así se mantiene al día con la API y otros desarrolladores saben cómo usarla sin leer el código.

3. Borrado de publicaciones y cascada

Petición	Esperado	Obtenido
DELETE /api/v1/posts/1 por el usuario B (no es el dueño)	403	403 Forbidden
DELETE /api/v1/posts/1 por el usuario A (dueño)	204 sin cuerpo	204 No Content, cuerpo vacío
GET /api/v1/posts/1 después de borrar	404	404 Not Found

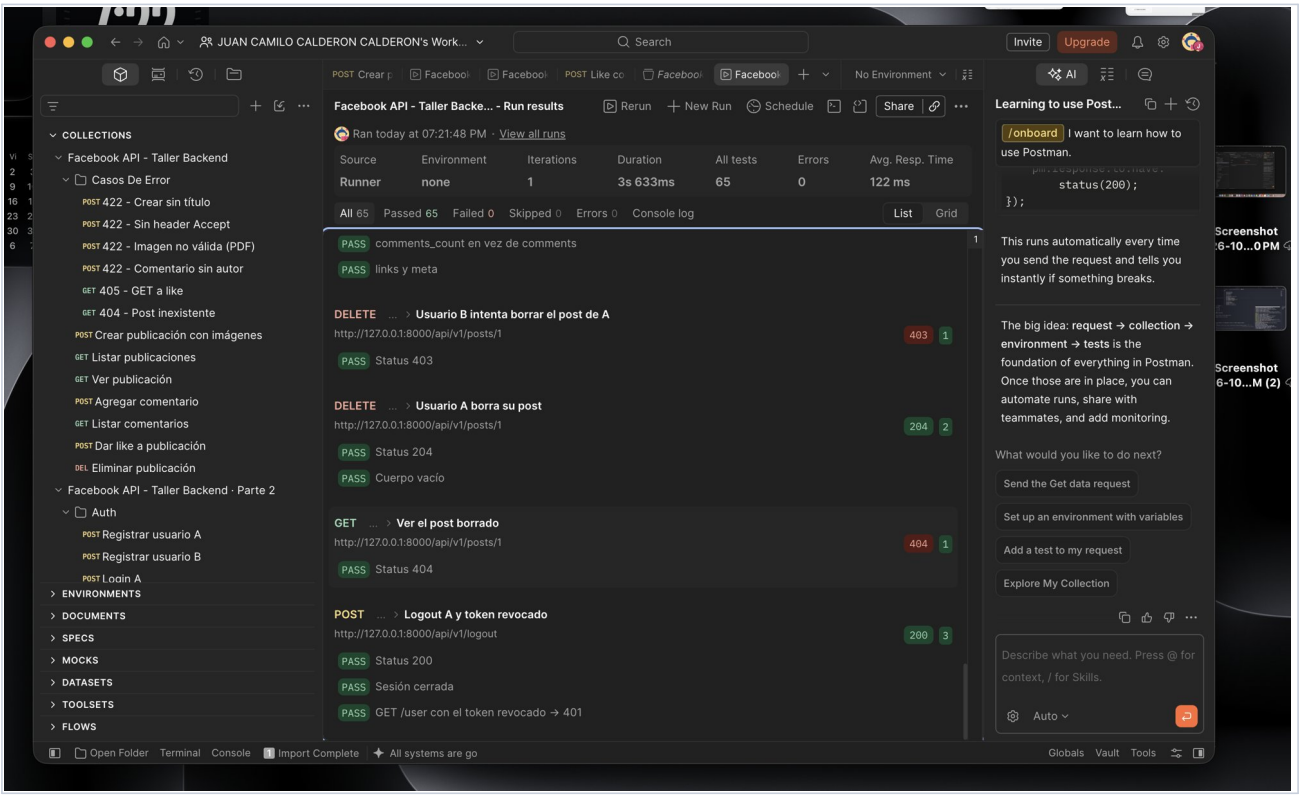


Figura 6. Collection Runner (corrida de las 7:21 PM, 65/65): v2 con comments_count, links y meta; Usuario B intenta borrar el post de A → 403; Usuario A borra su post → 204 ("Cuerpo vacío"); Ver el post borrado → 404; Logout A → 200 y GET /user con el token revocado → 401.

Estado después de correr la colección · fuente: 02-evidencia-terminal.txt

```
# Después de correr la colección (el post 1 fue borrado por su dueño):
$ ls -la storage/app/public/posts
total 0
drwxr-xr-x@ 2 camilocalderon  staff   64 Oct 12 17:52 .
drwxr-xr-x@ 4 camilocalderon  staff  128 Oct  8 11:56 ..
posts=10 post_images=0 comments=0 likes(post_user)=0
```

Durante la corrida, el post 1 se creó con 2 imágenes, recibió 2 likes (usuarios A y B) y 1 comentario. Después de que su dueño lo borró, la carpeta de imágenes quedó vacía (borrado físico de los archivos) y `post_images`, `comments` y `post_user` quedaron en 0 (borrado en cascada por las claves foráneas). Los 10 posts que siguen (`posts=10`) son los 10 extra creados para la paginación, que no tenían imágenes, comentarios ni likes.

¿Por qué? DELETE elimina el recurso y 204 indica éxito sin cuerpo; 403 protege el recurso de un usuario que no es el dueño, y 404 confirma que ya no existe.

4. Pruebas

Collection Runner de Postman: 65 pruebas, 65 aprobadas y 0 fallidas en las dos corridas capturadas (Figura 1 a las 5:51 PM y Figuras 3 y 6 a las 7:21 PM).

Newman (colección corrida por terminal) · fuente: 01-newman-resultado.txt

```
┌─────────────────────────┬────────────────────┬────────────────────┐
│                         │           executed │             failed │
├─────────────────────────┼────────────────────┼────────────────────┤
│              iterations │                  1 │                  0 │
│                requests │                 36 │                  0 │
│            test-scripts │                 25 │                  0 │
│      prerequest-scripts │                  1 │                  0 │
│              assertions │                 65 │                  0 │
├─────────────────────────┴────────────────────┴────────────────────┤
│ total run duration: 3s                                           │
│ average response time: 83ms [min: 4ms, max: 1532ms, s.d.: 255ms] │
└──────────────────────────────────────────────────────────────────┘
```

Las 25 test-scripts corresponden a los 25 requests de la colección; las peticiones HTTP son 36 porque "Crear 10 posts más" envía 10 y otros dos requests envían 2 cada uno.

Pruebas automáticas (PHPUnit) · fuente: 02-evidencia-terminal.txt

```
$ php artisan test
{"tool":"phpunit","result":"passed","tests":67,"passed":67,"assertions":299,"duration_ms":634}
```

5. Control de versiones

Commits de la Parte 2, del más reciente al más antiguo. Los 13 primeros salen de `git log --oneline parte-1..HEAD` (02-evidencia-terminal.txt); los 2 más recientes se hicieron después de ese registro y se ven en la Figura 7. 9 commits de la Parte 1 + 15 de la Parte 2 = 24, el total que muestra GitHub (Figura 8).

Commit	Mensaje (Conventional Commits)	Fuente
`1444a1f`	docs: actualizar progreso del paso 10	Figura 7
`c7cc62f`	test: exportar colección de Postman de la parte 2	Figura 7
`6940a80`	fix: corregir conteo de pruebas en README y agregar lista de evidencias	.txt
`576b74d`	docs: actualizar README con la parte 2	.txt
`51a6564`	test: completar pruebas de la parte 2	.txt
`81ebf64`	docs: documentar la API con Scribe	.txt
`df772d9`	fix: no fallar el comentario si el correo no se puede enviar	.txt
`85fe645`	feat: versionar la API en v1 y agregar v2	.txt
`e224793`	feat: notificar por correo al dueño del post al comentar	.txt
`149677a`	feat: buscar publicaciones por título o contenido	.txt
`b15fefe`	feat: paginar el listado de publicaciones	.txt
`87793bb`	feat: likes únicos por usuario con tabla pivote	.txt
`5f32dad`	feat: proteger endpoints y asociar publicaciones y comentarios al usuario	.txt
`b71a2bd`	feat: agregar registro e inicio de sesión con Sanctum	.txt
`e25d482`	docs: agregar guía de la parte 2 del taller	.txt

Archivos sensibles fuera del repositorio · fuente: 02-evidencia-terminal.txt

```
$ git ls-files | grep -E "\.env$|sqlite"   (vacío = no se versionan)
(vacío)
```

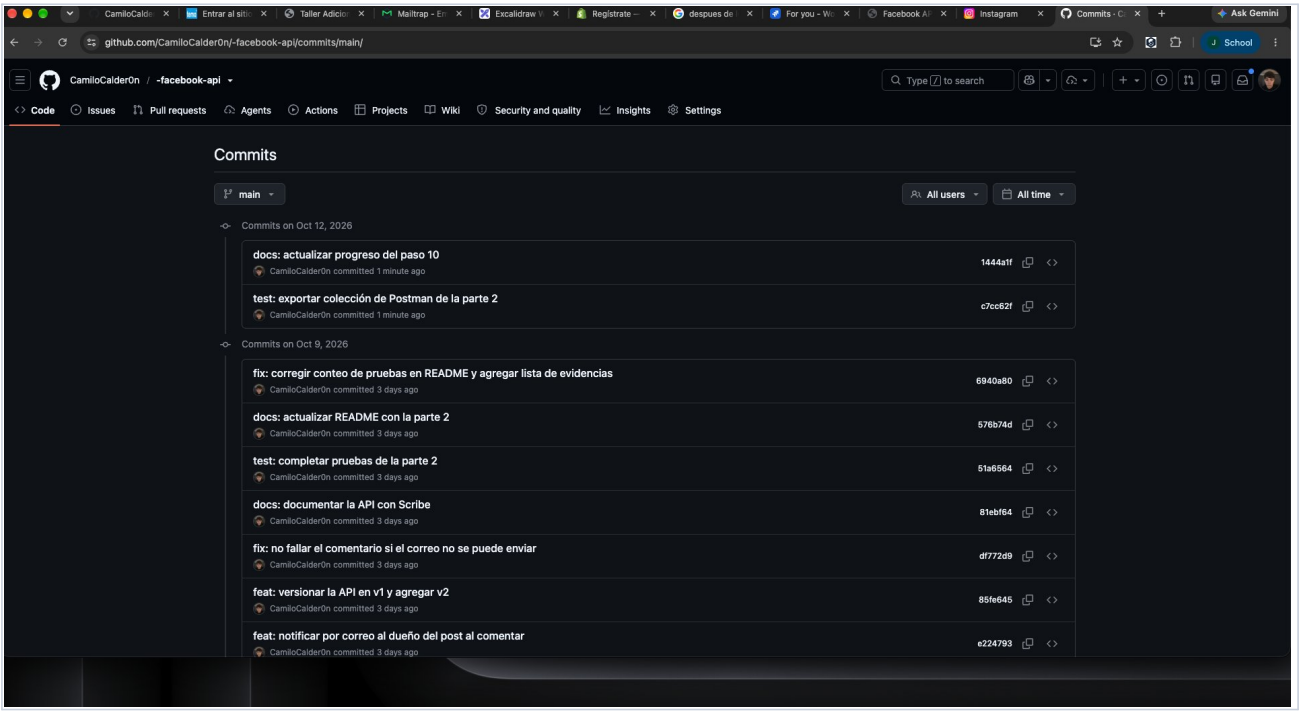


Figura 7. GitHub, historial de la rama main: commits del 12 y del 9 de octubre de 2026 con mensajes en Conventional Commits en español (docs:, test:, fix:, feat:).

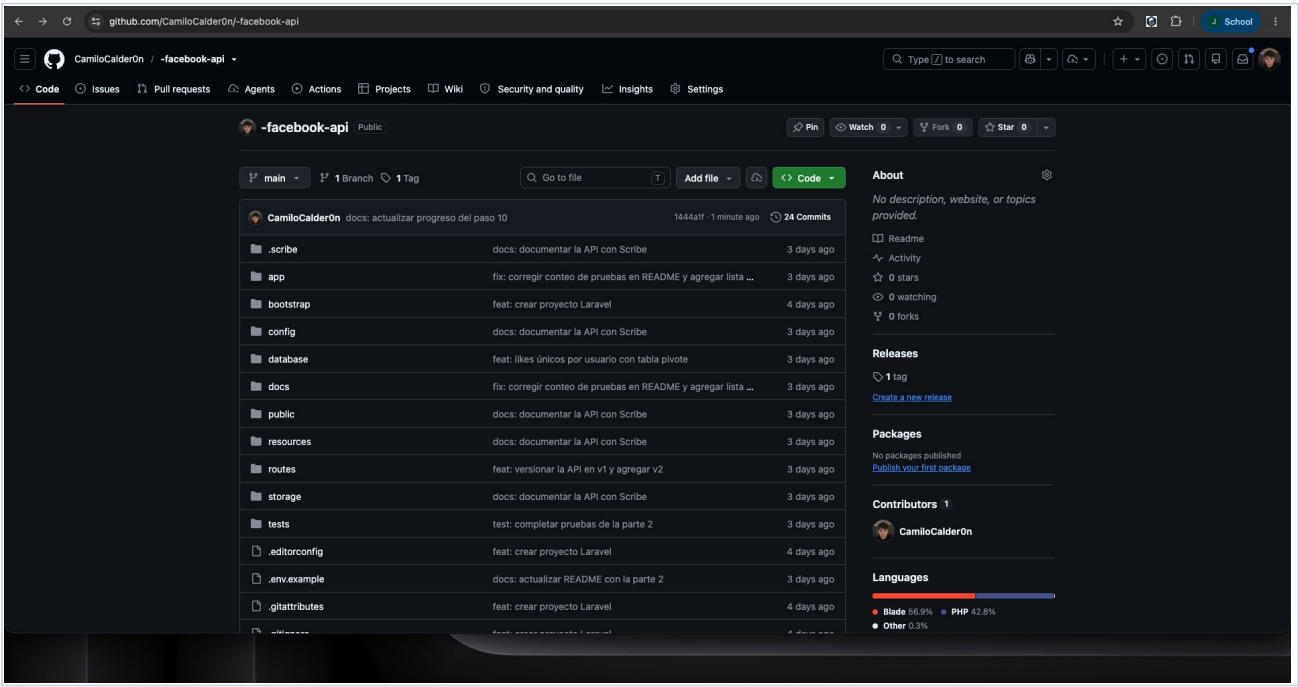


Figura 8. GitHub: repositorio público "-facebook-api", rama main, 1 tag y 24 commits; último commit 1444a1f "docs: actualizar progreso del paso 10".

6. Checklist final

Requisito	Evidencia	Estado
Registro e inicio de sesión con tokens (Sanctum)	Figura 1; Newman (sección 2, Reto 1)	**Cumplido**
Rutas de escritura protegidas: sin token → 401	Newman; Figuras 3 y 2	**Cumplido**
Publicaciones y comentarios asociados al usuario autenticado	Newman ("Guarda user_id", "author es el usuario B"); migraciones add_user_id	**Cumplido**
Solo el dueño puede borrar su publicación (403)	Figura 6; Newman	**Cumplido**
Logout revoca el token (401 al reutilizarlo)	Figuras 6 y 2; Newman	**Cumplido**
Paginación de GET /posts (?page=1, ?page=2)	Newman (Reto 2)	**Cumplido**
Likes únicos por usuario con tabla pivote (200 → 409 → 200)	Figura 3; Newman; migración post_user	**Cumplido**
Búsqueda por título o contenido (?search=)	Newman (Reto 4)	**Cumplido**
Correo "Nuevo comentario" al dueño del post	Newman (201); Figura 4 (Mailtrap)	**Cumplido**
Versionado: /api/v1, alias /api y /api/v2	Newman; route:list (Reto 6)	**Cumplido**
Documentación de la API con Scribe (/docs)	Figura 5	**Cumplido**
Borrado: 204, 404 posterior, archivos y cascada	Figura 6; 02-evidencia-terminal.txt (sección 3)	**Cumplido**
Colección de Postman: 65/65 en Runner y en Newman	Figuras 1, 3 y 6; 01-newman-resultado.txt	**Cumplido**
Pruebas automáticas: 67/67 en verde	02-evidencia-terminal.txt (sección 4)	**Cumplido**
Commits de la Parte 2 en Conventional Commits y tag parte-1	Figuras 7 y 8; git log parte-1..HEAD	**Cumplido**
.env y database.sqlite fuera del repositorio	02-evidencia-terminal.txt (sección 5)	**Cumplido**